COMP0222 Simultaneous Localisation and Mapping

Lecture 13: Point Cloud SLAM

Simon Julier

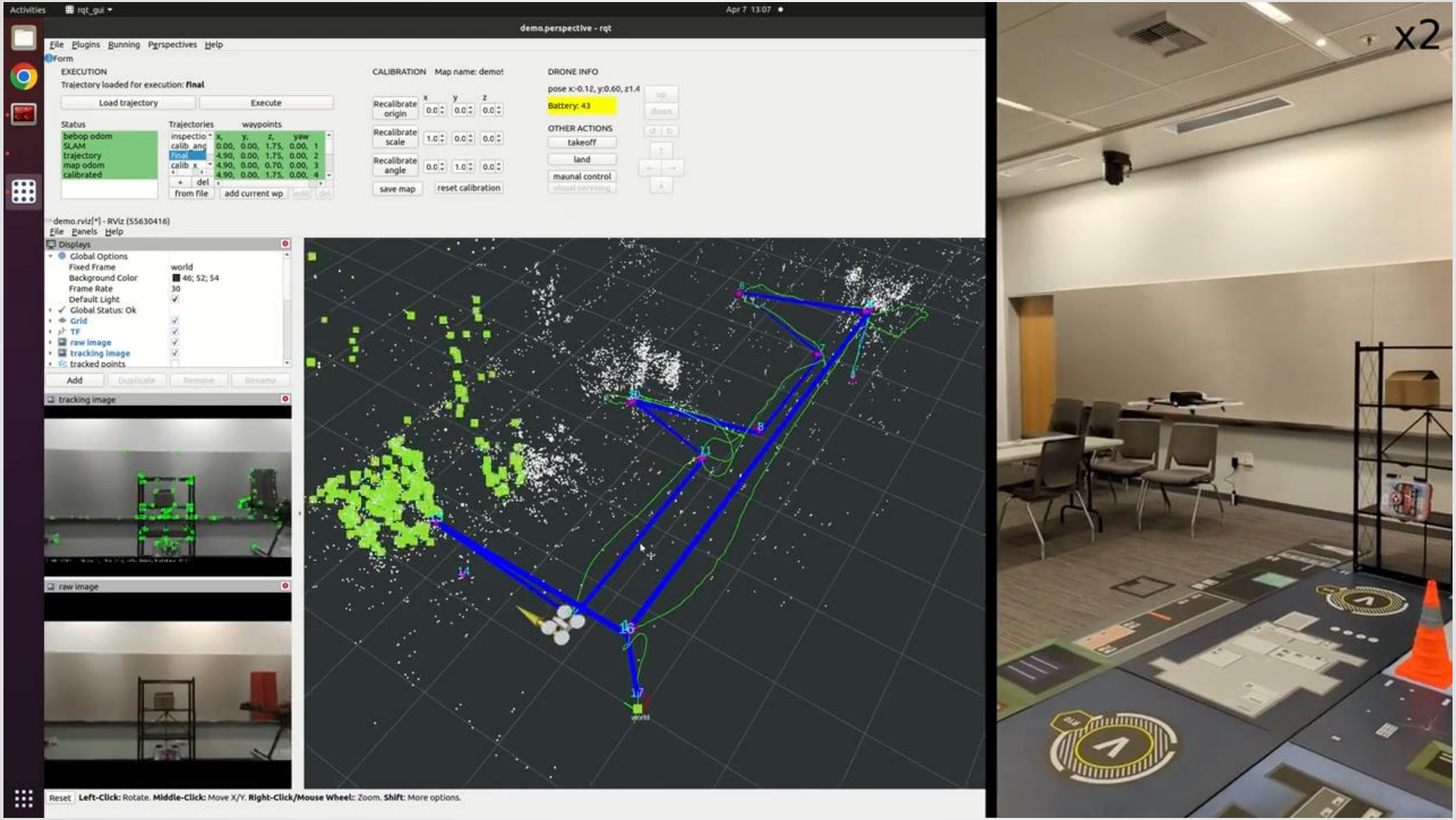
s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

Strengths and Weaknesses of Sparse Feature Maps

- So far we've looked at ways to build maps of *sparse* feature points
- These are very effective for localizing the platform in real-time

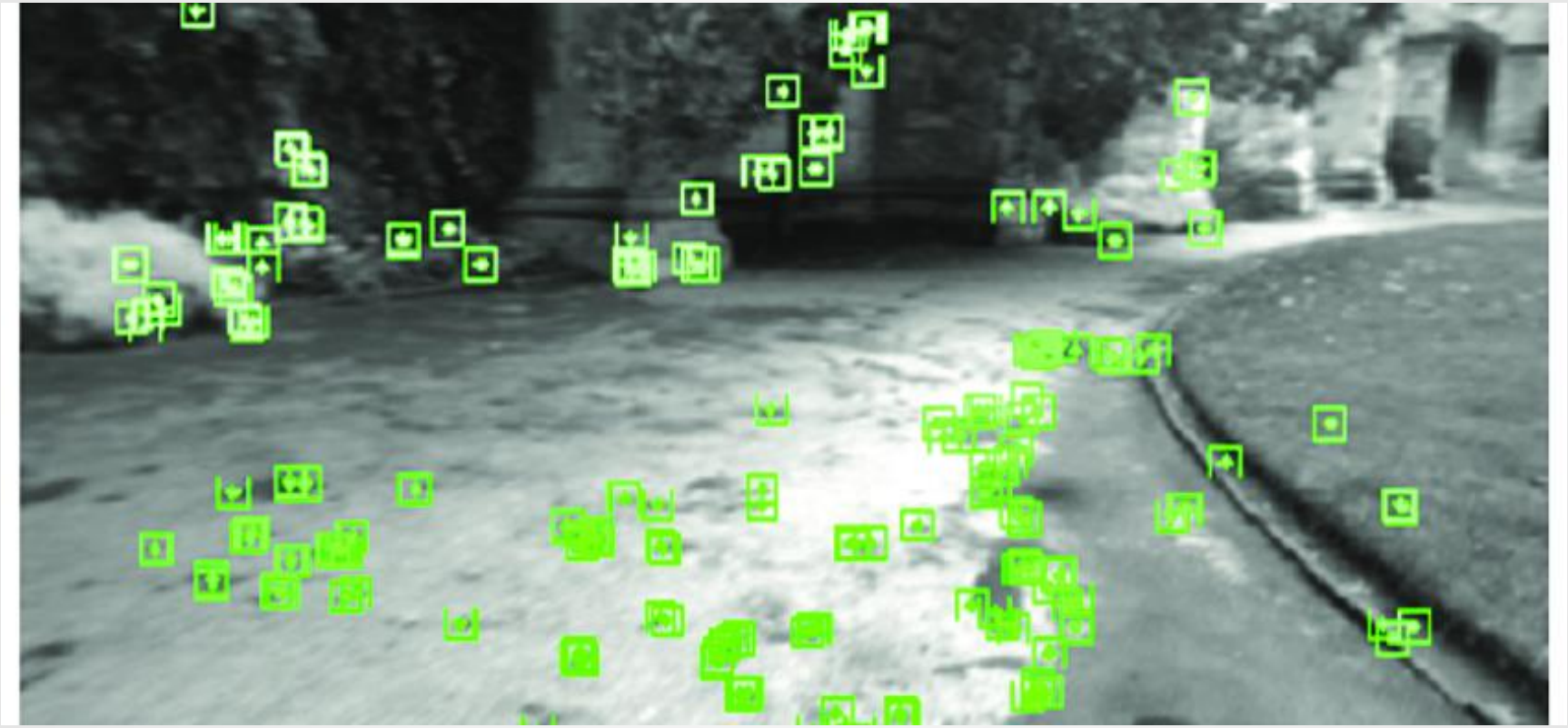


From "Indoor drone navigation using Monocular Orb Slam 3 and ROS", A. Bucker

Strengths and Weaknesses of Sparse Feature Maps

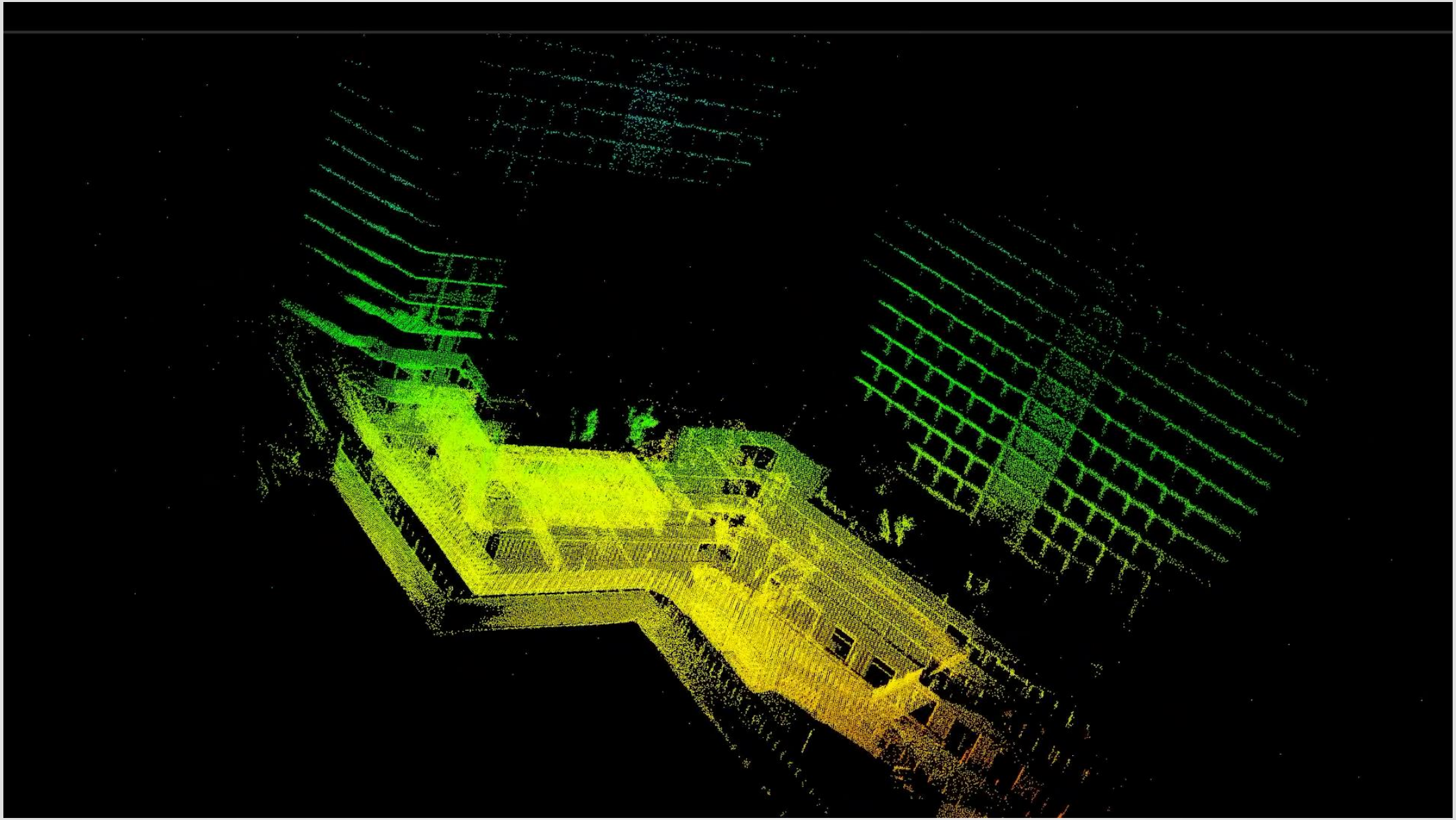
- So far we've looked at ways to build maps of *sparse* feature points
- These are very effective for localizing the platform in real-time
- However, they don't tell us much about the *structure* of the environment

Limitations of Sparse Feature Maps



Strengths and Weaknesses of Sparse Feature Maps

- So far we've looked at ways to build maps of *sparse* feature points
- These are very effective for localizing the platform in real-time
- However, they don't tell us much about the *structure* of the environment
- There are lots of tasks where knowing the structure is useful
- In this lecture we'll look at point clouds and SLAM



3D Point Cloud of One Pool Street Using Fast-LIO SLAM

Why Do We Need Point Clouds?

- Why Do We Need Point Clouds?
- Depth Sensing Systems
- Overview of the Point Cloud SLAM
- Feature-Based Scan Matching
- Iterative Closest Point Scan Matching
- Point-to-Plane and Generalized ICP

Why Do We Need Point Clouds?

- **Why Do We Need Point Clouds?**
- ***Depth Sensing Systems***
- ***Overview of the Point Cloud SLAM***
- ***Feature-Based Scan Matching***
- ***Iterative Closest Point Scan Matching***
- ***Point-to-Plane and Generalized ICP***

Point Clouds

- A point cloud is a set of 3D points in space:

$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

- However, unlike sparse points we do not estimate the state of each point in the cloud separately
- Rather, we estimate the position of the sensor and back project to construct the cloud

Why Can't We Just Use Sparse Features?

- **Point clouds looks a lot like sparse features:**
  - The map is composed of lots of individual points in 3D space
  - These points are referenced using keyframes

- **However, there are two big differences:**
  - Scalability
  - The types of sensors used

Kenilworth COLMAP 3D Model

Scalability

- This model took *five and a half hours* to build
- The frame-to-frame association used sequential matching and only took *a few minutes*
- The rest of the time was taken by the optimization used in the bundle adjustment
- Therefore, a model with this level of complexity cannot be built in real-time using sparse feature SLAM

Types of Sensors Used

- We want to have sensors which can *recover the complete structure* of the environment
- This includes areas with plain walls or very repetitive textures

Highly Confusing Textures

Types of Sensors Used

- We want to have sensors which can recover the complete structure of the environment
- This includes areas with plain walls or very repetitive textures
- You either can't find image-based features, or you find a huge number of them which look very similar to one another
- Therefore, you need to use other sensing systems to guarantee robustness
- The ones we are interested

Depth Sensing Systems

- *Why Do We Need Point Clouds?*
- Depth Sensing Systems
- *Overview of the Point Cloud SLAM*
- *Feature-Based Scan Matching*
- *Iterative Closest Point Scan Matching*
- *Point-to-Plane and Generalized ICP*

Depth Sensing Systems

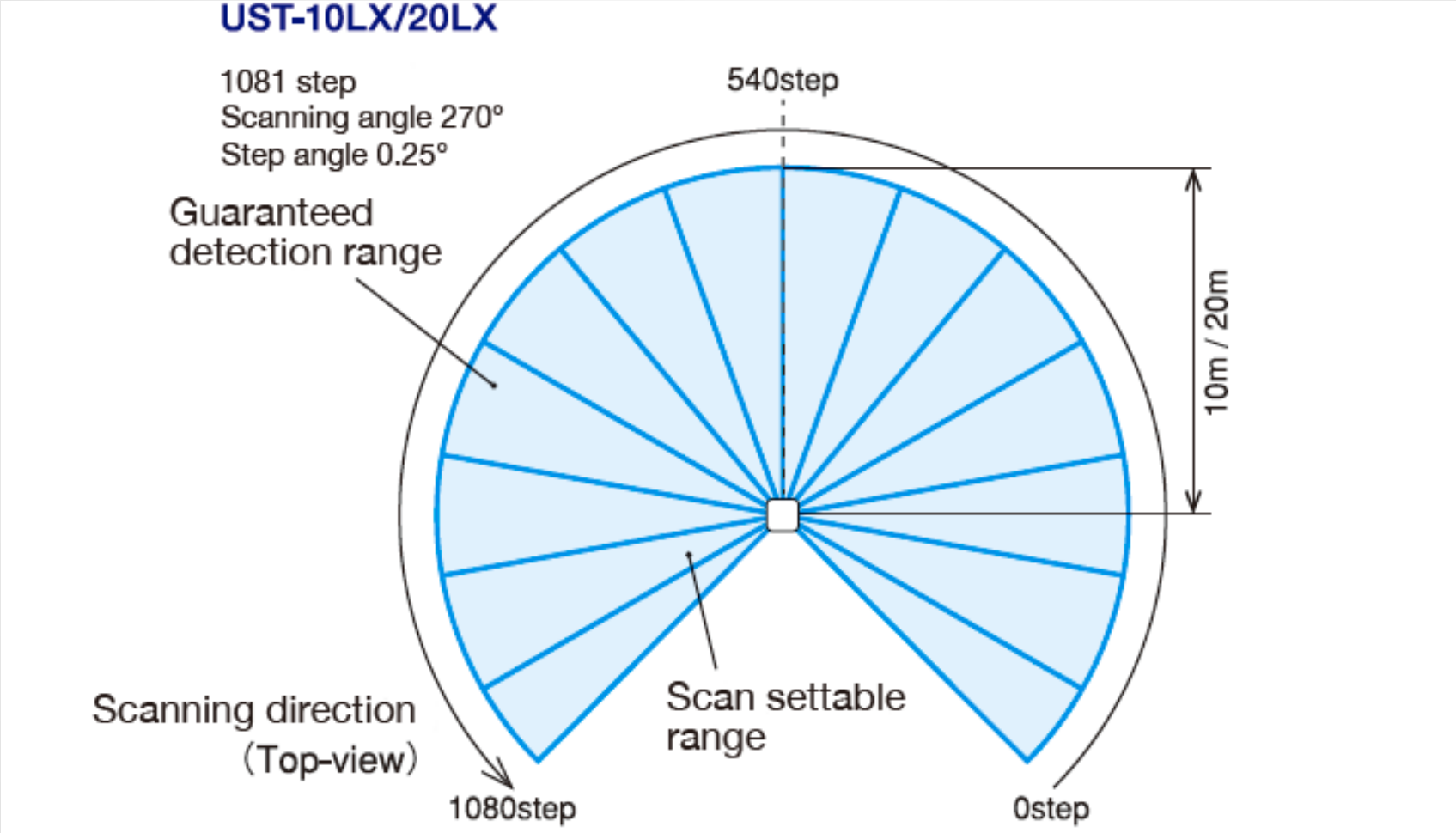
- Depth sensing systems directly measure the depth to features in the environment
- This means we don’t just have to rely on visually distinct features

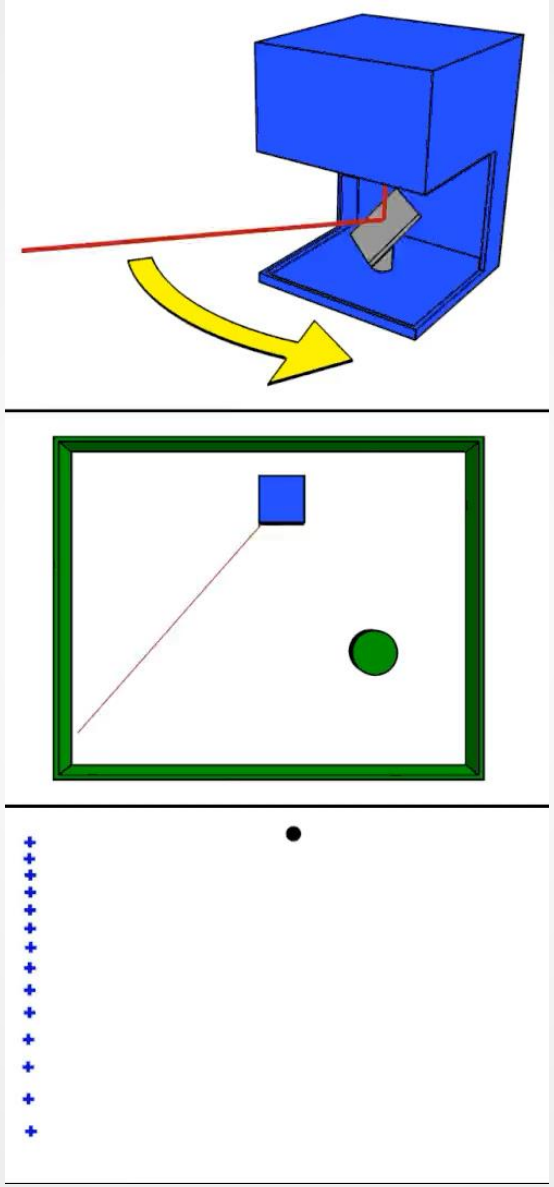
Textureless Walls

Depth Sensing Systems

- Depth sensing systems directly measure the depth to features in the environment
- This means we don’t just have to rely on visually distinct features
- We don’t have to worry about things like scale drift
- Common device types are LiDAR and Radar
- By default these tend to operate in 2D

2D LiDAR Scan





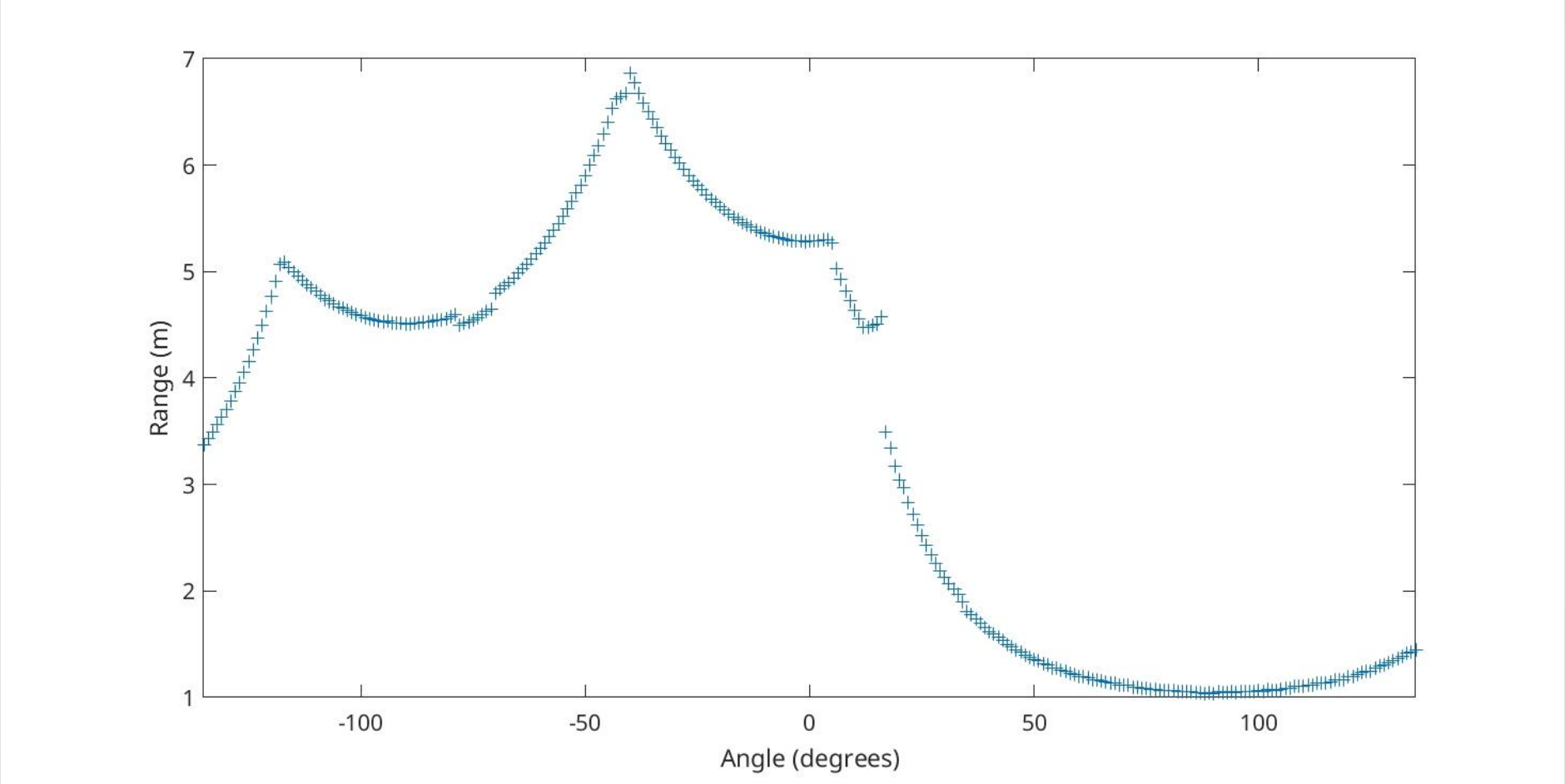
2D LiDAR Scan

- **Each scan consists of a set of range bearing measurements,**

$$S_k = \left\{ \mathbf{s}_k^1, \ldots, \mathbf{s}_k^{M_k} \right\}$$

$$\mathbf{s}_k^j = \begin{pmatrix} r_k^j \\ \beta_k^j \end{pmatrix}$$

Example 2D LiDAR Scan



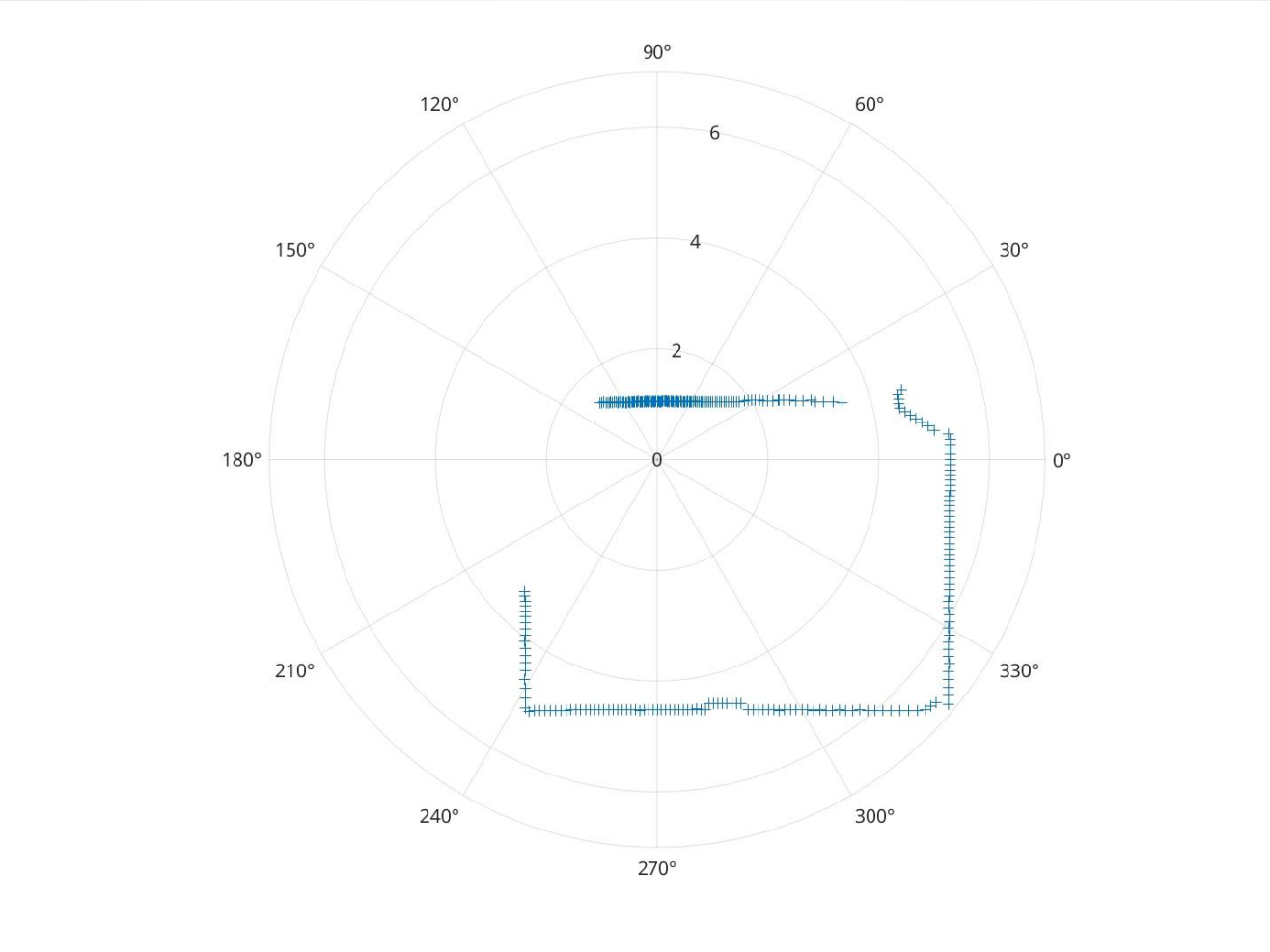
2D LiDAR Scan Cloud

- **These measurements are normally converted into a set of points in the sensor reference frame,**

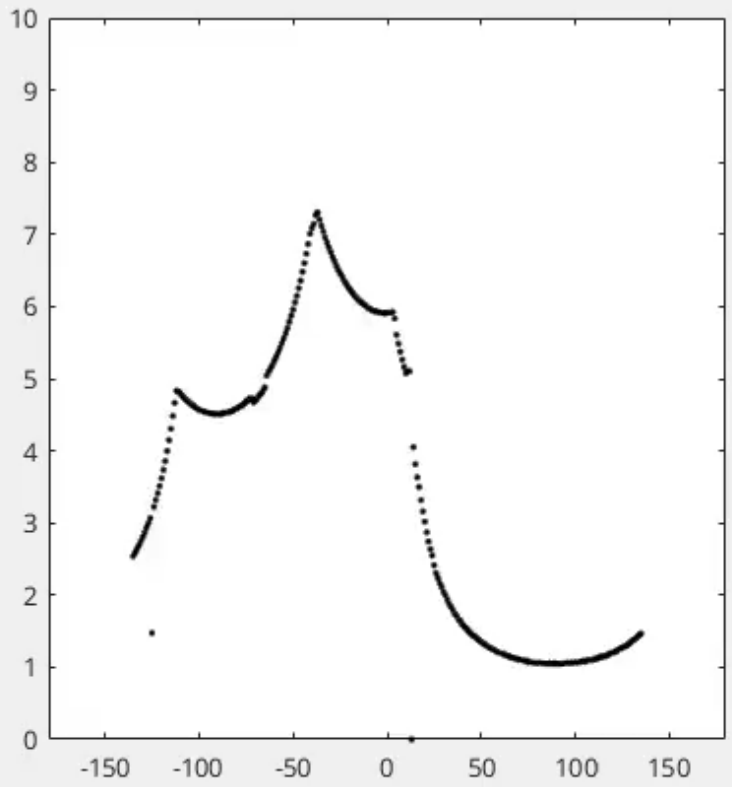
$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

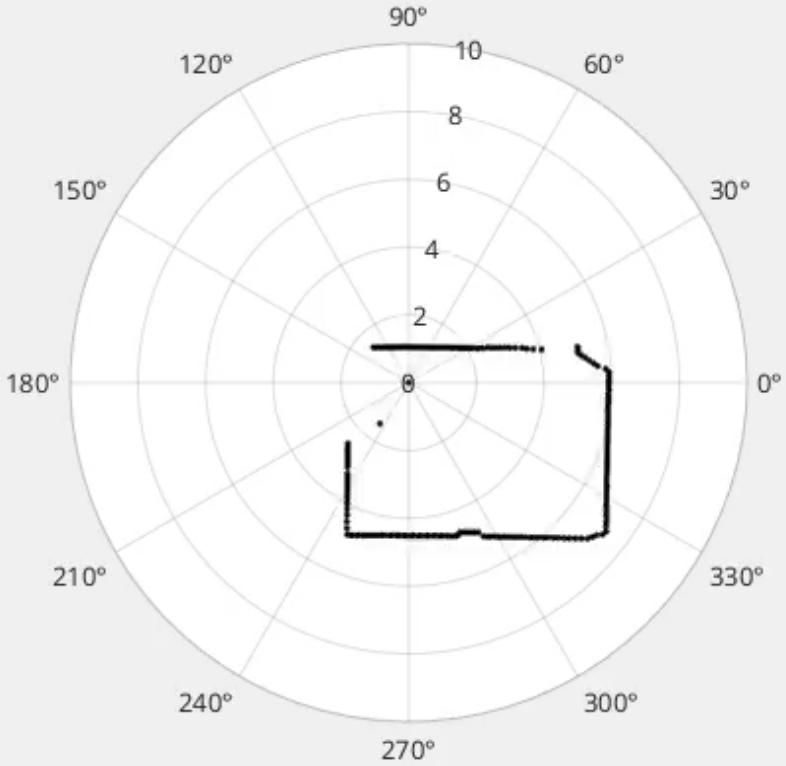
$$\mathbf{o}_k^j = r_k^j \begin{pmatrix} \cos \beta_k^j \\ \sin \beta_k^j \end{pmatrix}$$

2D LiDAR Scan Cloud



2D LiDAR Scan and Scan Cloud Over Time



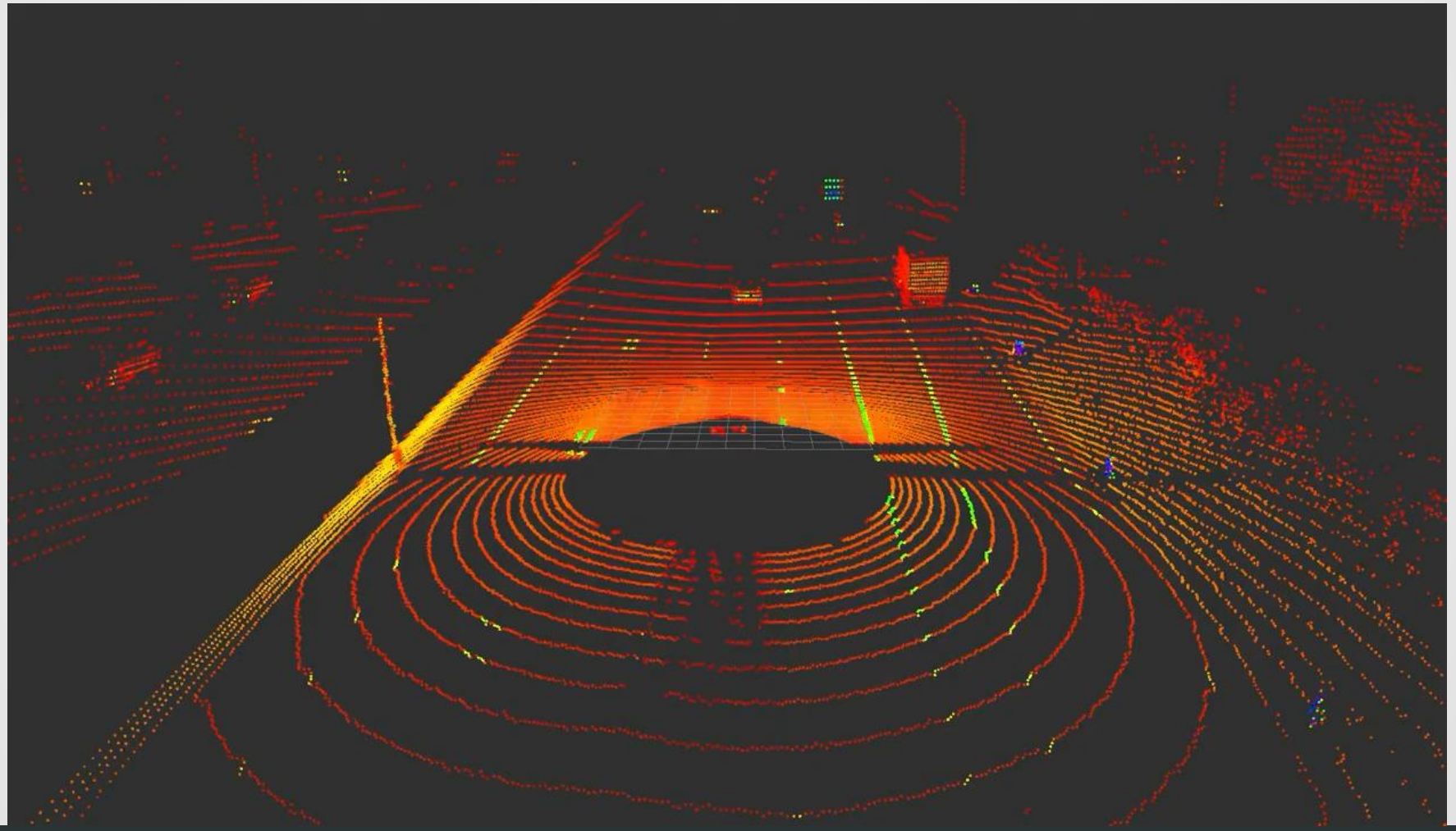


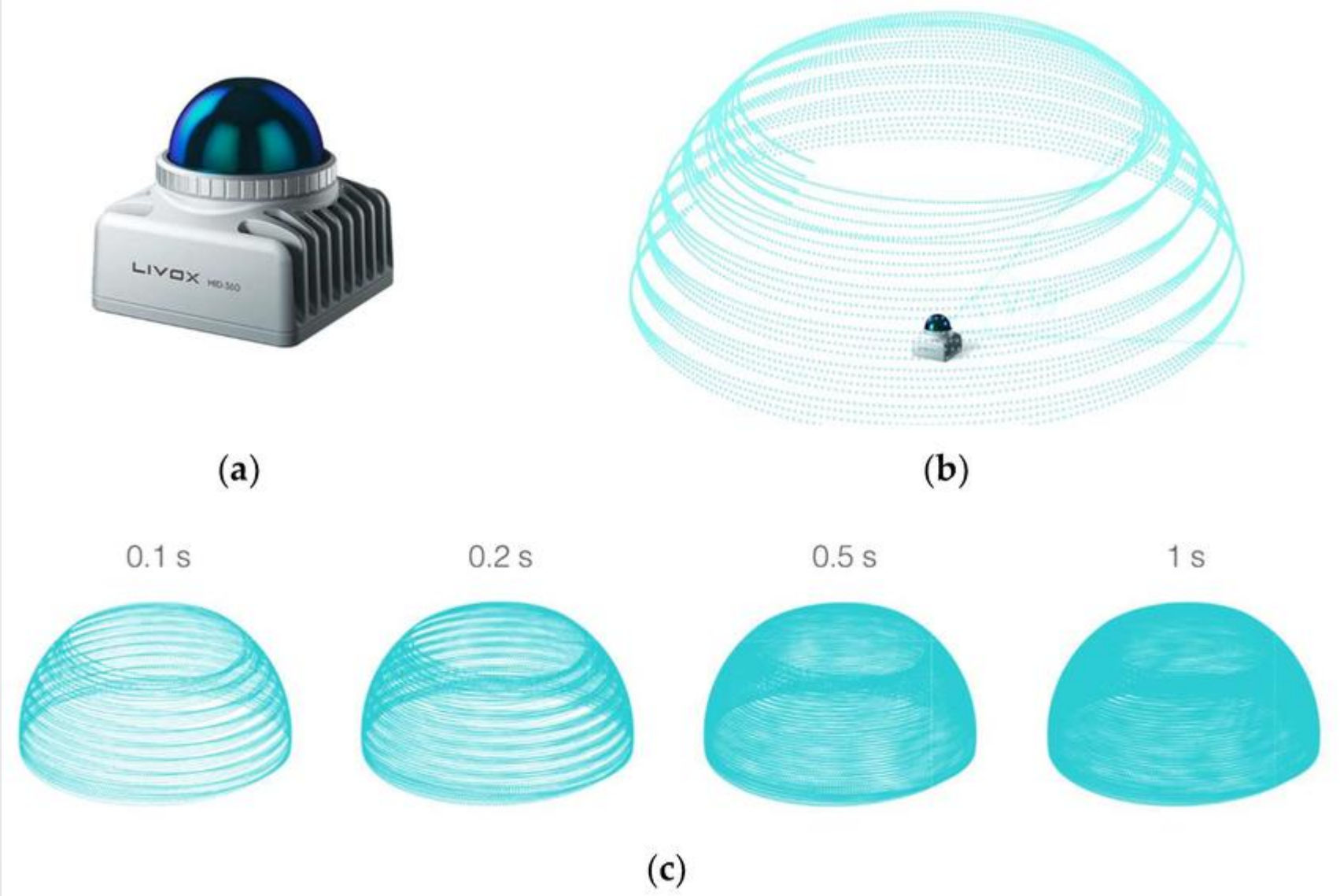
3D LiDAR

- LiDAR systems can be very accurate and large range
- However, most LiDAR systems are 2D
- 3D is usually obtained using multiple LiDAR and / or scanning patterns

3D LiDAR (Ouster OS1-32)

Ouster used in Self-Driving Application

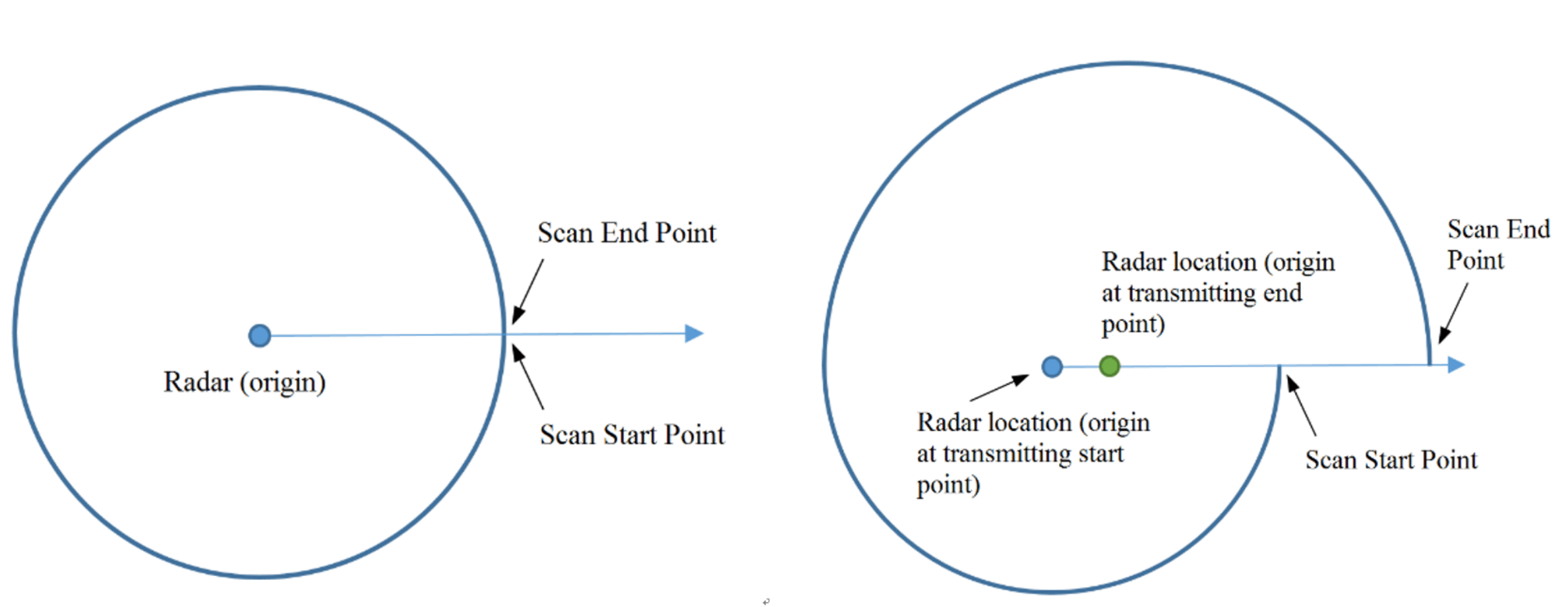



LIVOX
(a)
(b)
0.1 s
0.2 s
0.5 s
1 s
(c)

Need for Motion Compensation

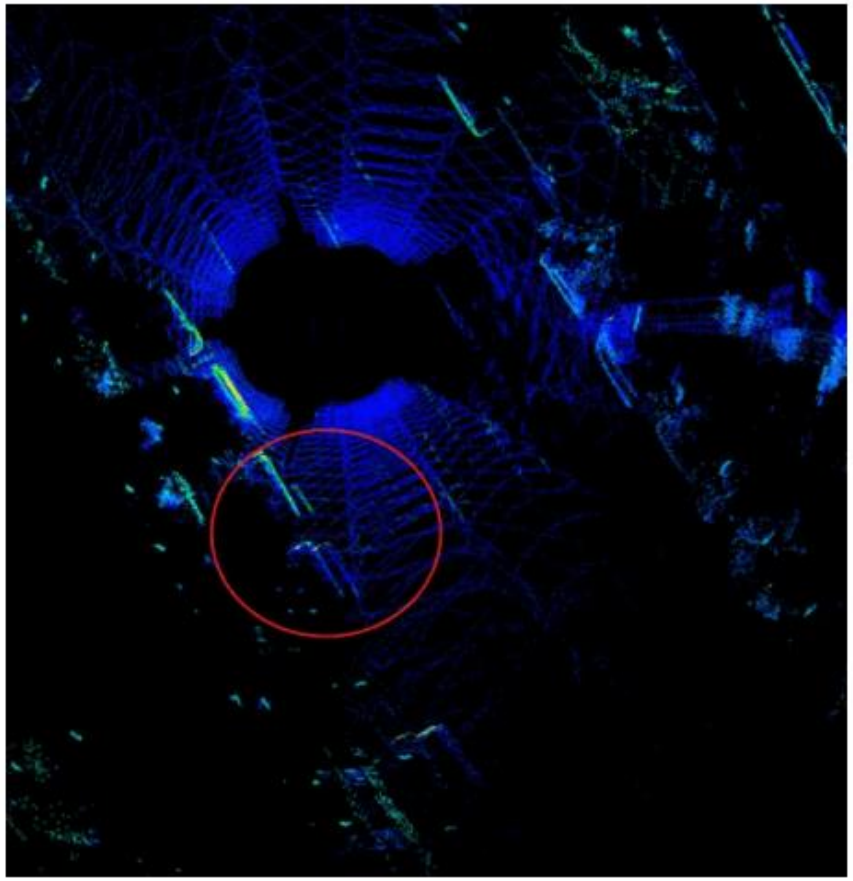
- Unlike a camera, a LiDAR system takes time to complete a scan
- During this time, the platform can move by a non-trivial amount
- As a result, platform movement distorts the LiDAR data

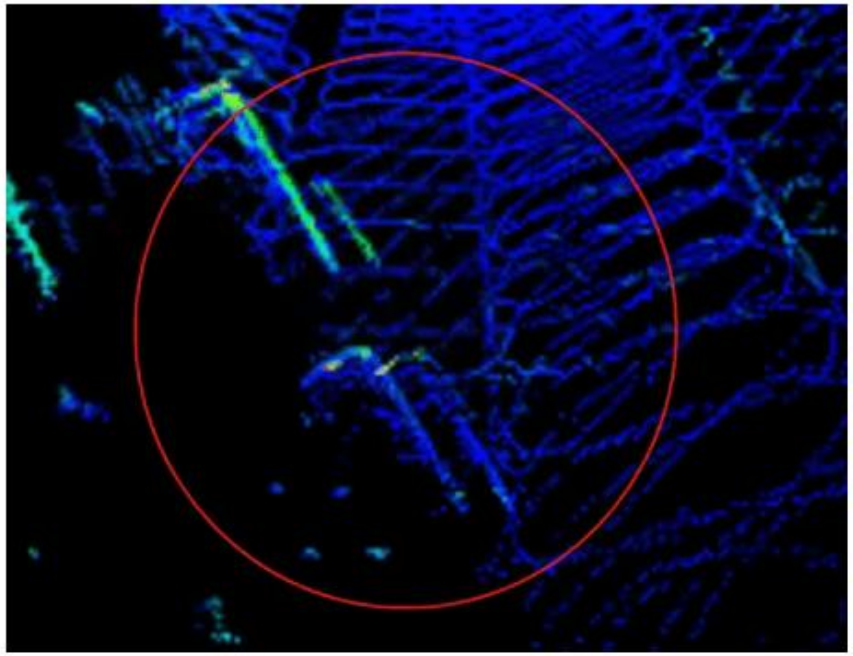
Need for Motion Compensation



From "Livox Open Source Sharing: About LiDAR Distortion Removal", 2021

Need for Motion Compensation

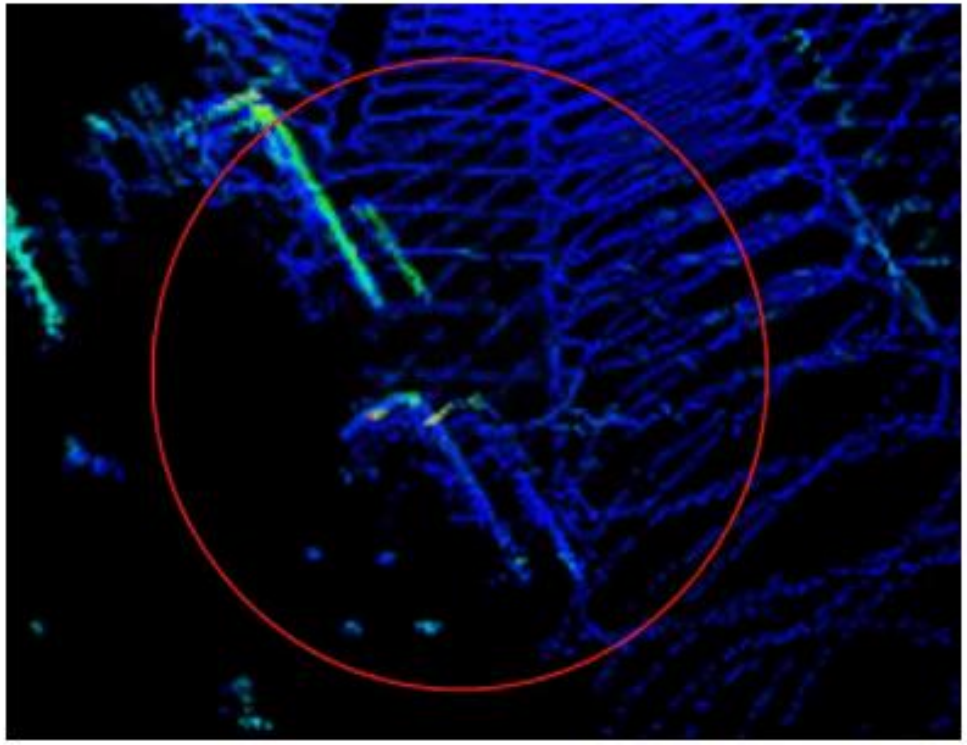


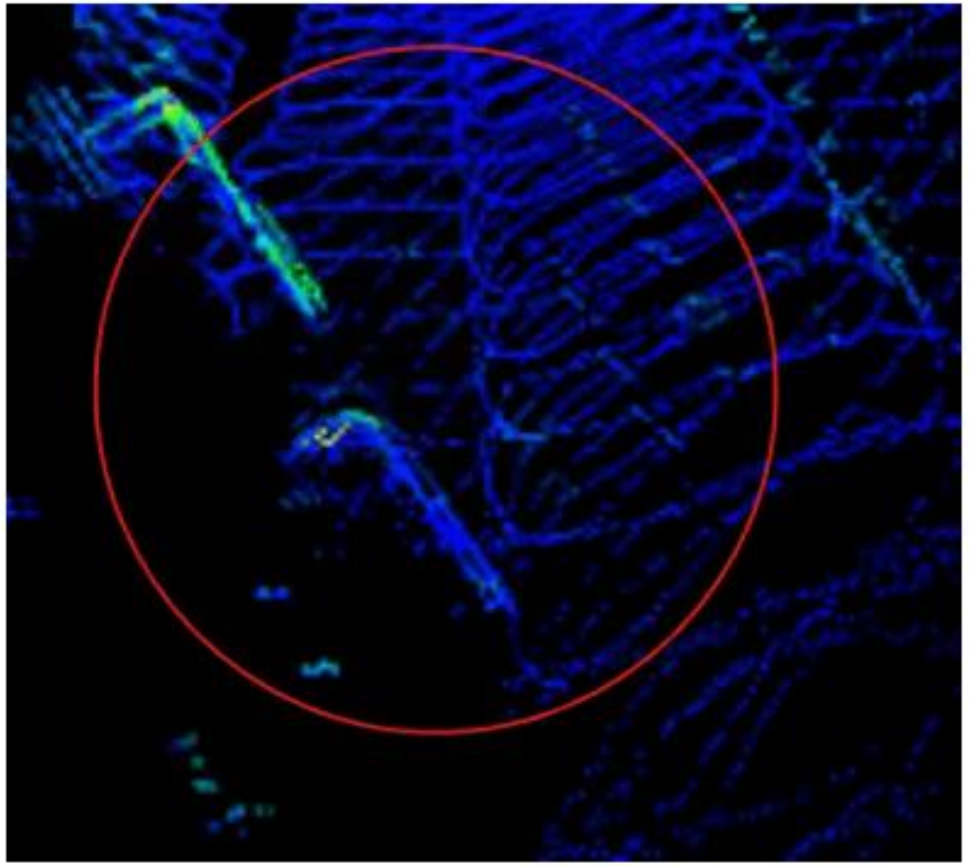


From "Livox Open Source Sharing: About LiDAR Distortion Removal", 2021

LiDAR Motion Compensation

Need for Motion Compensation



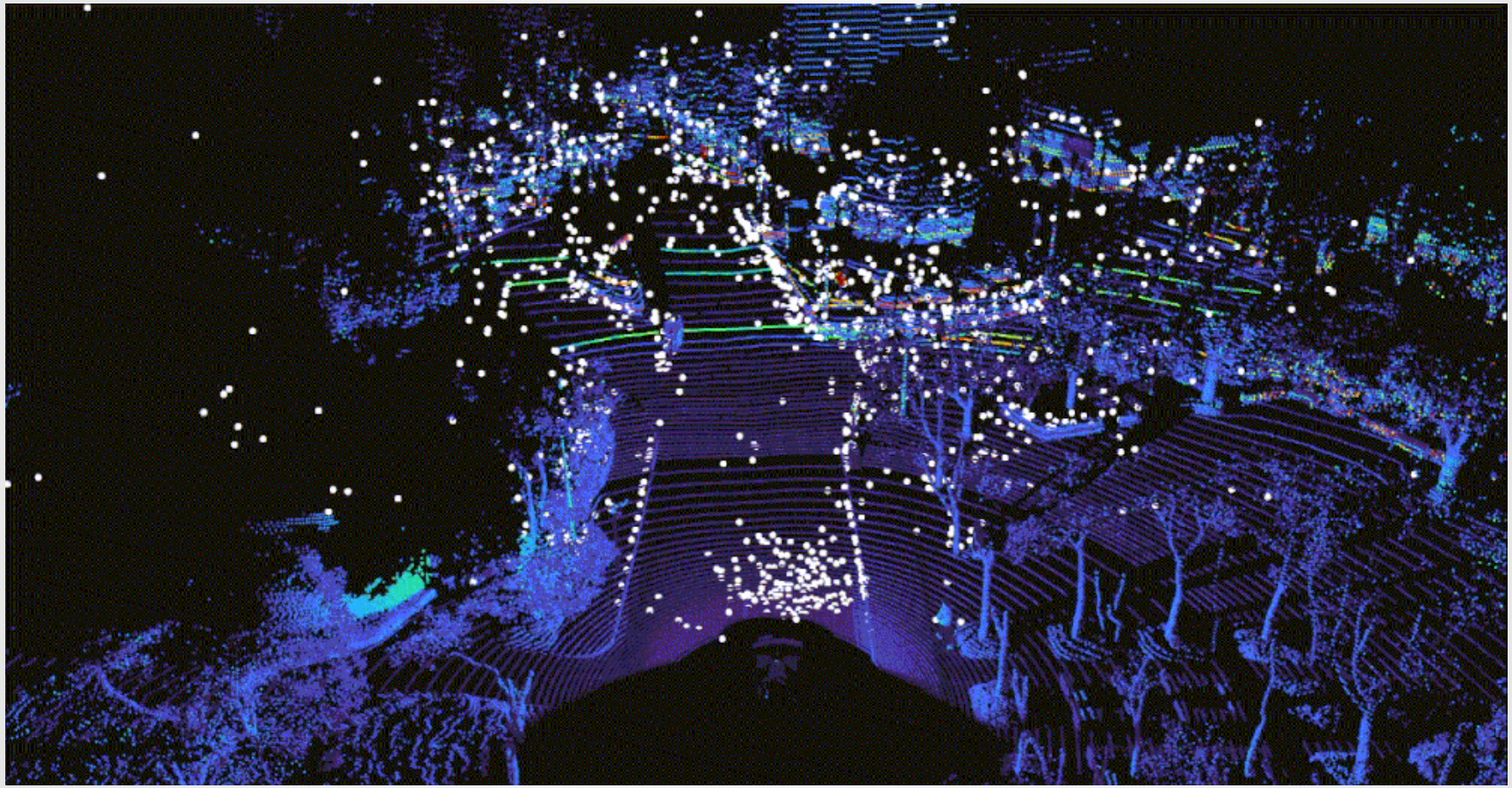


From "Livox Open Source Sharing: About LiDAR Distortion Removal", 2021

RADAR

- LiDAR systems rely on optical wavelengths
- Therefore, they can be obscured by dust and smoke
- RADAR systems, which use longer wavelengths can penetrate through dust and smoke

Radar



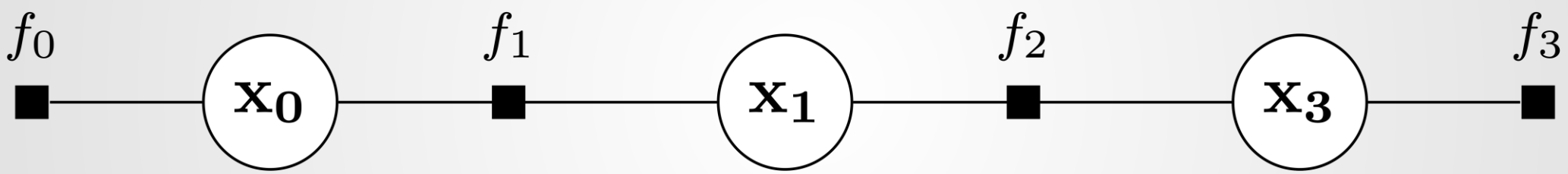
Overview of the Point Cloud SLAM

- *Why Do We Need Point Clouds?*
- *Depth Sensing Systems*
- Overview of the Point Cloud SLAM
- *Feature-Based Scan Matching*
- *Iterative Closest Point Scan Matching*
- *Point-to-Plane and Generalized ICP*
- *Loop Closure*

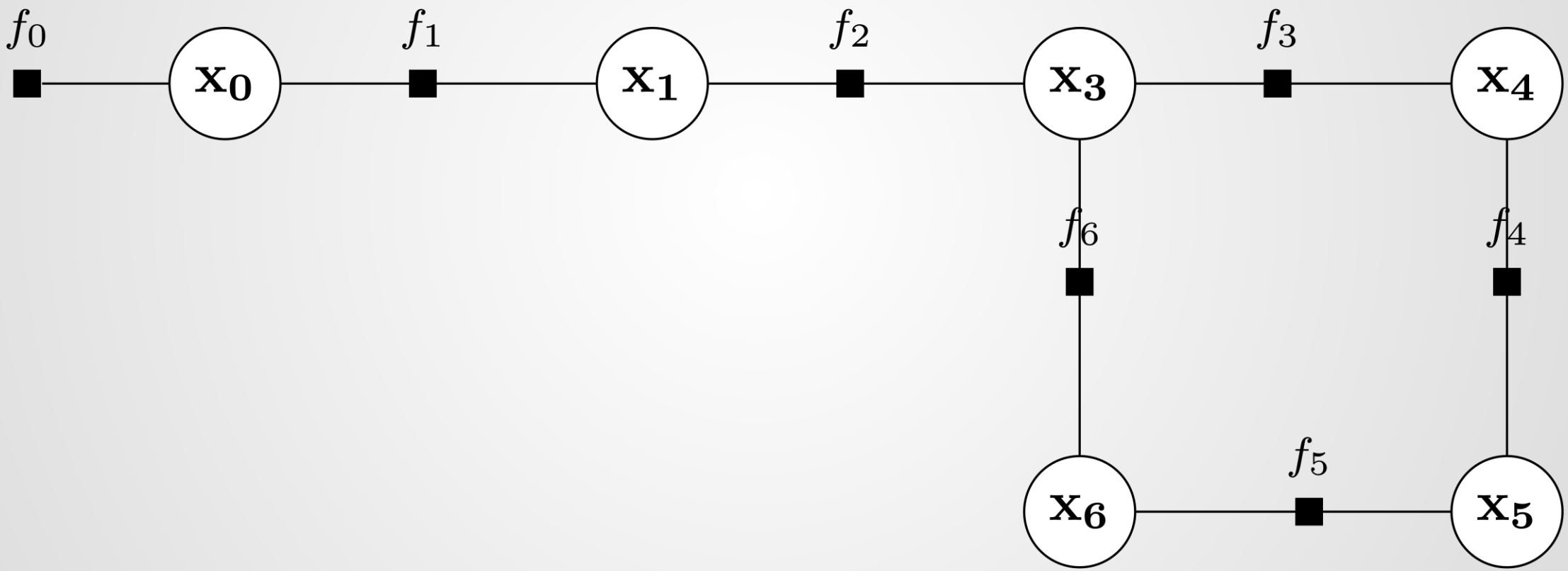
Point Cloud SLAM

- There are many implementations of point clouds in SLAM
- Most of them resemble the Essential Graph in ORB-SLAM:
  - The only vertex type is the pose of a keyframe
  - There is an initial pose edge to initialize
  - There is a relative pose edge to compute the relative transformation between keyframes

Structure of the Factor Graph



Structure of the Factor Graph with Loop Closing



Keyframe Vertices

- **Each keyframe stores an estimate of the pose (position, orientation), and velocity (linear, angular)**

$$\mathbf{x}_k = \begin{bmatrix} \mathbf{R}_k \\ \mathbf{p}_k \\ \dot{\mathbf{p}}_k \end{bmatrix}$$

Keyframe Vertices

- **The transformation matrix which specifies the transformation from the origin to the pose at k is given by**

$$\mathbf{T}_k$$

Keyframe Vertices

- **Each keyframe also stores the associated LiDAR point cloud**

$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

Initial Condition Edge

- This is the same as with regular SLAM
- It specifies a prior over the position, orientation and velocity of the platform

Transformation Edges

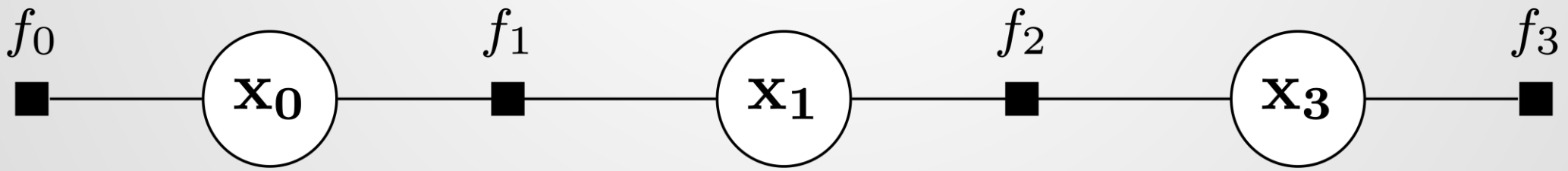
- **The transformation or odometry edges specify the transformation between two platform poses and velocities**

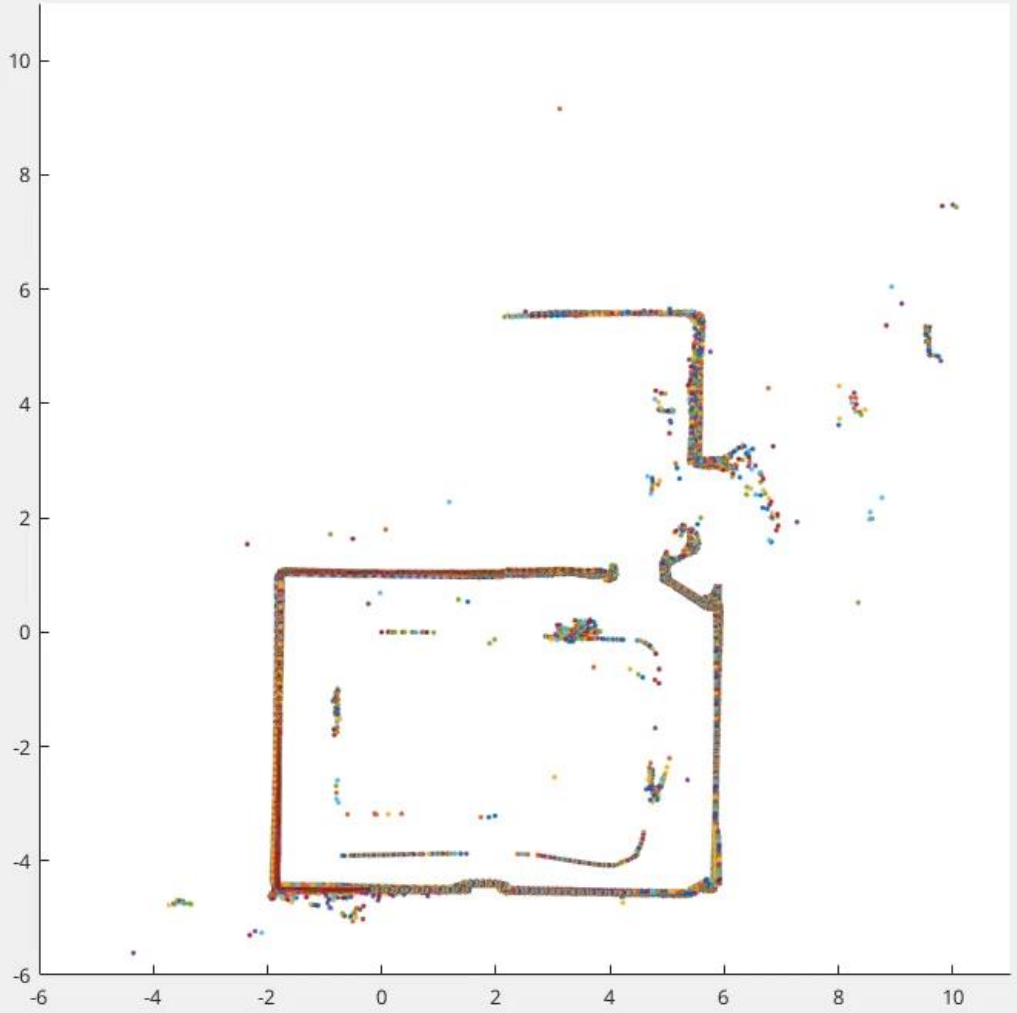
$$\Delta \mathbf{T}_{k,k+1} = \mathbf{T}_k^\top \mathbf{T}_{k+1}$$

Hang On… Where's the Map?

- Where is it?
- We don't estimate it directly
- Rather, we construct the estimates of the point clouds by back projecting the local cloud back into the world coordinate system
- This back projection can also be used to perform loop closing

Projecting the Point Cloud to Make the Map




10
8
6
4
2
0
-2
-4
-6
-6
-4
-2
0
2
4
6
8
10

Operations

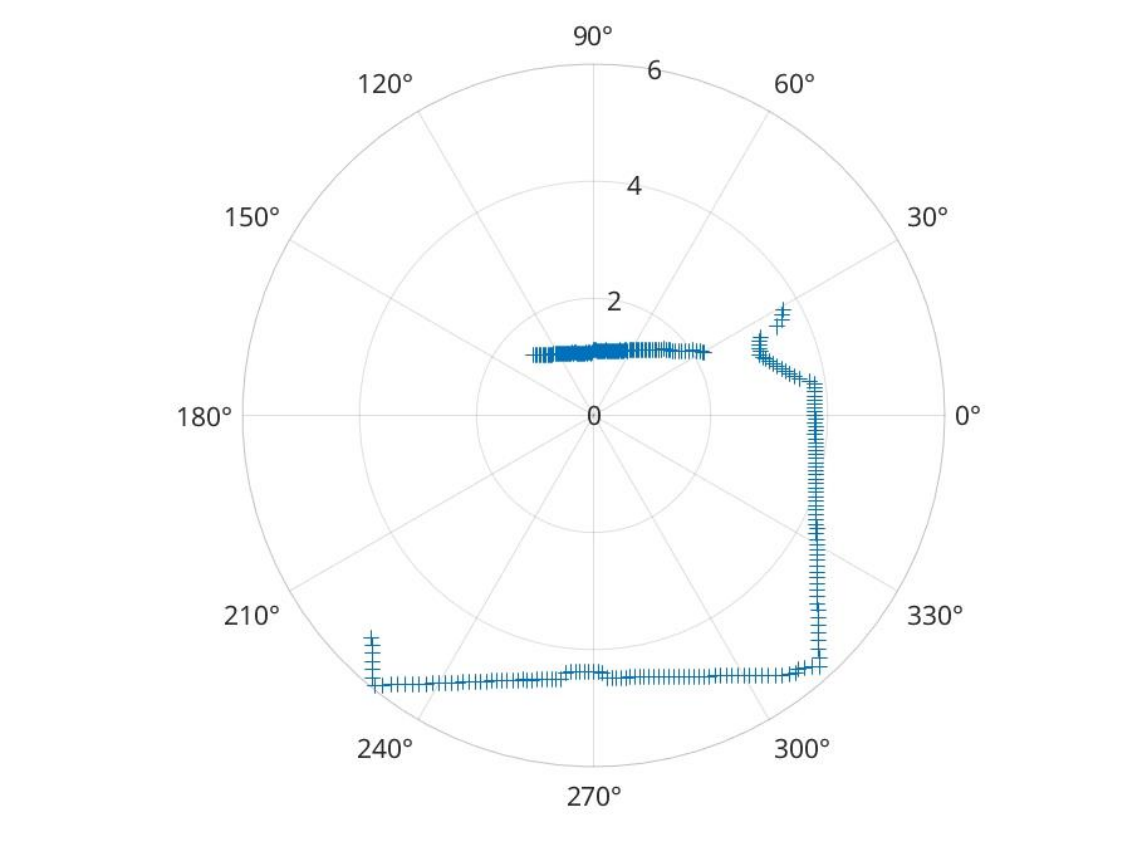
- We have several things we need to do:
  - How can we estimate the odometry from one frame to the next?
  - How can we do loop closure?

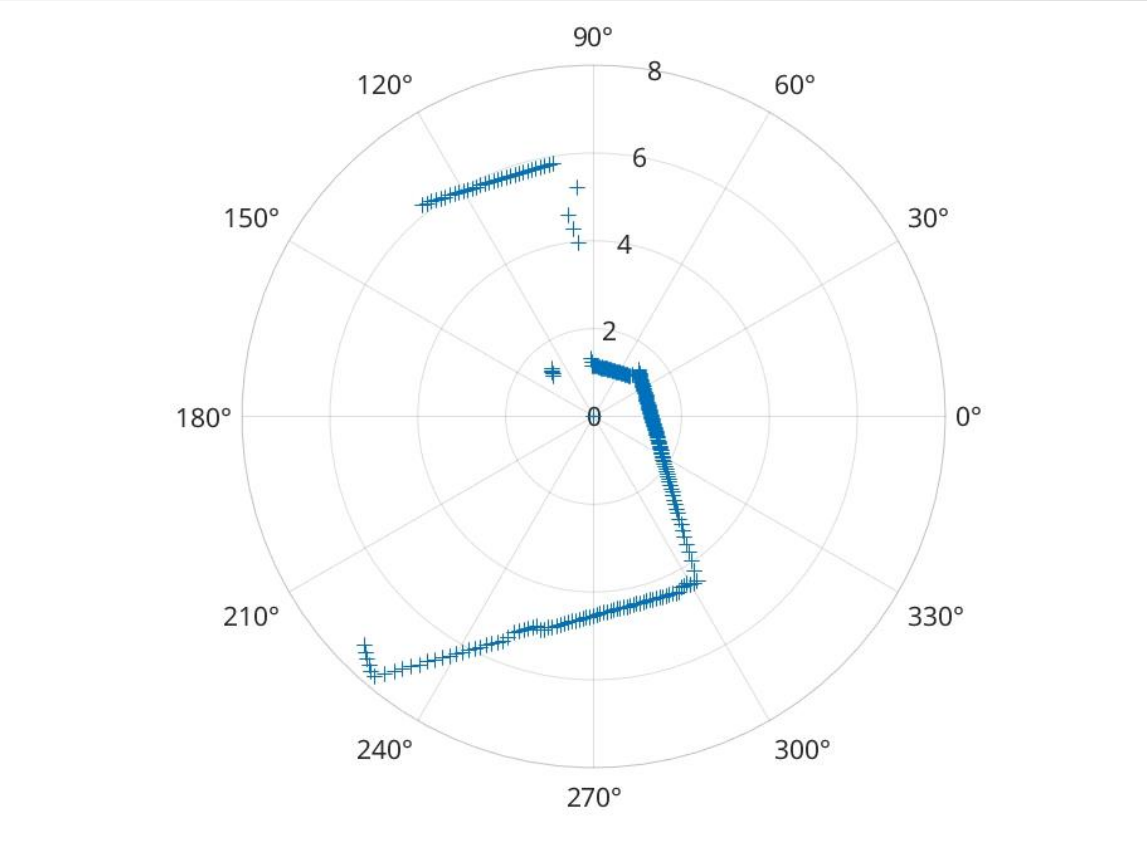
- Both of these require us to *align or register scans*
- We'll now look at the scan alignment process

Feature-Based Scan Matching

- *Why Do We Need Point Clouds?*
- *Depth Sensing Systems*
- *Overview of the Point Cloud SLAM*
- **Feature-Based Scan Matching**
- *Iterative Closest Point Scan Matching*
- *Point-to-Plane and Generalized ICP*
- *Loop Closure*

Feature-Based Registration Problem





Registration Problem

- Suppose we have two scans O_k and O_{k+1} taken of the same scene from the platform at two different times

$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

$$O_{k+1} = \left\{ \mathbf{o}_{k+1}^1, \ldots, \mathbf{o}_{k+1}^{M_{k+1}} \right\}$$

Registration of Registered Points

- We want to work out the relative transformation from the platform at k to the platform at $k+1$
- This is given by the equation,

$$\Delta \mathbf{T}_{k,k+1} = \mathbf{T}_k^\top \mathbf{T}_{k+1}$$

- This is transformation which can be parameterized by the translation vector $\Delta \mathbf{t}_{k,k+1}$ and the rotation matrix $\Delta \mathbf{M}_{k,k+1}$

Registration Problem

- **Therefore, the relationship between the points in the two scans is given by**

$$\mathbf{o}_{k+1}^{j} = \Delta\mathbf{M}_{k,k+1}\mathbf{o}_{k}^{j} + \Delta\mathbf{t}_{k,k+1}$$

Association Vector

- **We need to define the association vectors**

$$I_k = \left\{ i_k^1,\ i_k^2,\ \ldots,\ i_k^{C_k} \right\}$$

$$I_{k+1} = \left\{ i_{k+1}^1,\ i_{k+1}^2,\ \ldots,\ i_{k+1}^{C_k} \right\}$$

Association Vector

$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

$$I_k = \left\{ i_k^1,\ i_k^2,\ \ldots,\ i_k^{C_k} \right\}$$

$$O_{k+1} = \left\{ \mathbf{o}_{k+1}^1, \ldots, \mathbf{o}_{k+1}^{M_{k+1}} \right\}$$

$$I_{k+1} = \left\{ i_{k+1}^1,\ i_{k+1}^2,\ \ldots,\ i_{k+1}^{C_k} \right\}$$

Registration Problem

- **The solution is given by**

$$\Delta \mathbf{M}_{k,k+1}, \Delta \mathbf{t}_{k,k+1} = \arg\min_{\mathbf{M},\mathbf{t}} \sum_{j=1}^{M_k} ||\mathbf{o}_{k+1}^{j} - \mathbf{M}\mathbf{o}_{k}^{j} - \mathbf{t}||^2$$

Closed Form Solution

- The alignment uses the Umeyama Alignment Algorithm we saw in evo
- It consists of four steps:
  - Work out the centroid of the two point clouds
  - Work out the cross correlation matrix between the point clouds
  - Use singular value decomposition to estimate rotation
  - Use the rotation to work out the final translation

Centroid Calculation

- **The centroid of each cloud is given by computing the average of the common points,**

$$\bar{\mathbf{o}}_k = \frac{1}{C_k} \sum_{j=1}^{C_k} \mathbf{o}_k^j, \quad \bar{\mathbf{o}}_{k+1} = \frac{1}{C_k} \sum_{j=1}^{C_k} \mathbf{o}_{k+1}^j$$

Cross Correlation Matrix

- **The cross correlation matrix is**

$$\boldsymbol{\Delta}_{k,k+1} = \frac{1}{C_k} \sum_{j=1}^{C_k} \left(\mathbf{o}_k^j - \bar{\mathbf{o}}_k \right) \left(\mathbf{o}_{k+1}^j - \bar{\mathbf{o}}_{k+1} \right)^\top$$

SVD and Rotation

- **Taking the singular value decomposition of the cross-correlation matrix,**

$$\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\top}=\boldsymbol{\Delta}_{\boldsymbol{k},\boldsymbol{k}+\mathbf{1}}$$

- **The rotation matrix is given by**

$$\mathbf{M}_{\boldsymbol{k}+\mathbf{1}}=\mathbf{V}\mathbf{U}^{\top}$$

Point Cloud Translation

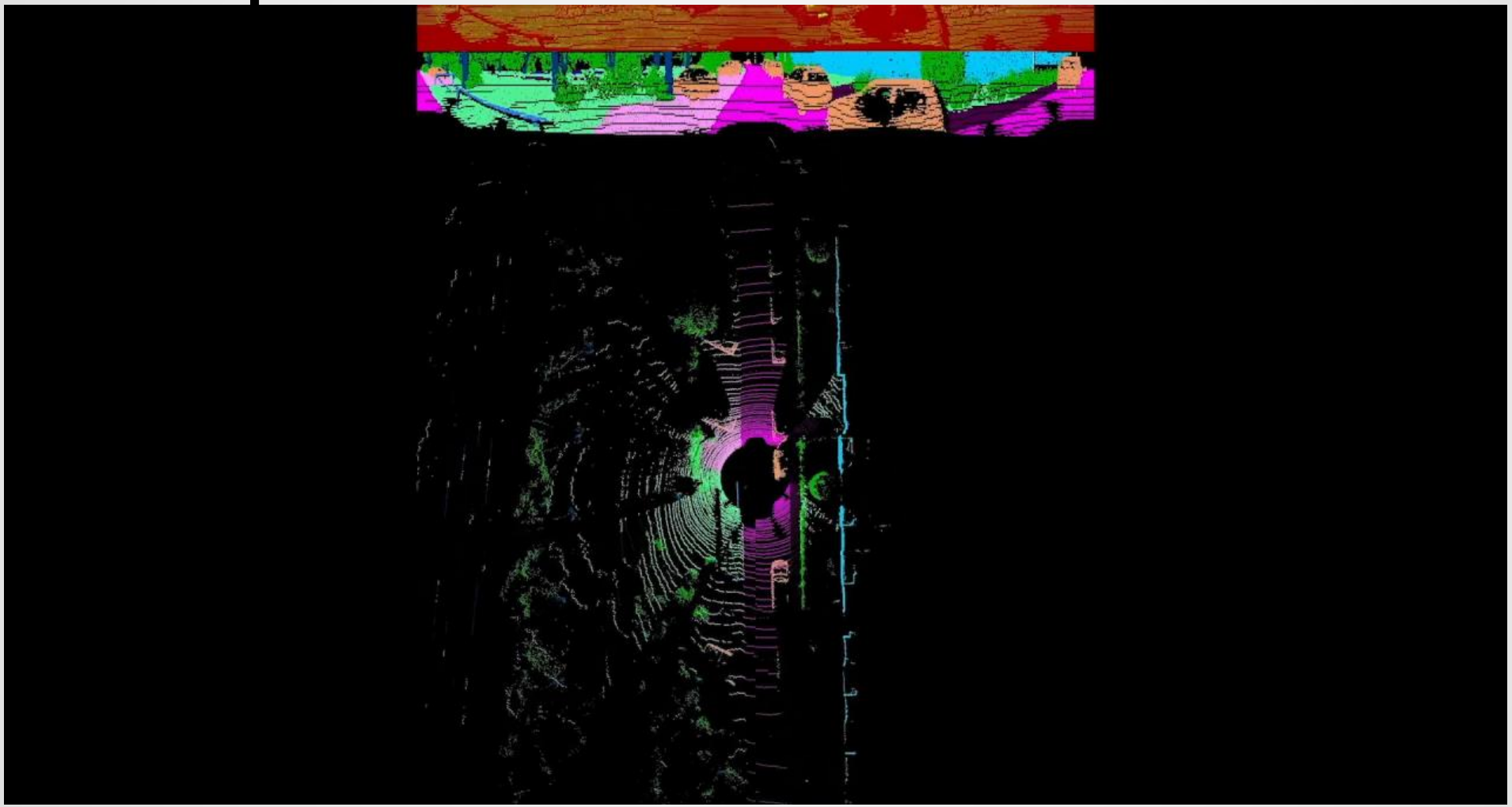
- **The translation is given by**

$$\mathbf{t}_{k+1} = \bar{\mathbf{o}}_{k+1} - \mathbf{M}_{k+1}\bar{\mathbf{o}}_k$$

Non-Matching Features

- The analysis so far has assumed that we have features which are common between the two scans
- However, this isn't guaranteed to be the case in general
- For example:
  - We could operate in environments without distinct features
  - We could operate in environments where the points don't even uniquely map to the same place

Lack of Uniqueness of Features



From "Laser Focused: How Multi-View LidarNet Presents Rich Perspective for Self-Driving Cars"

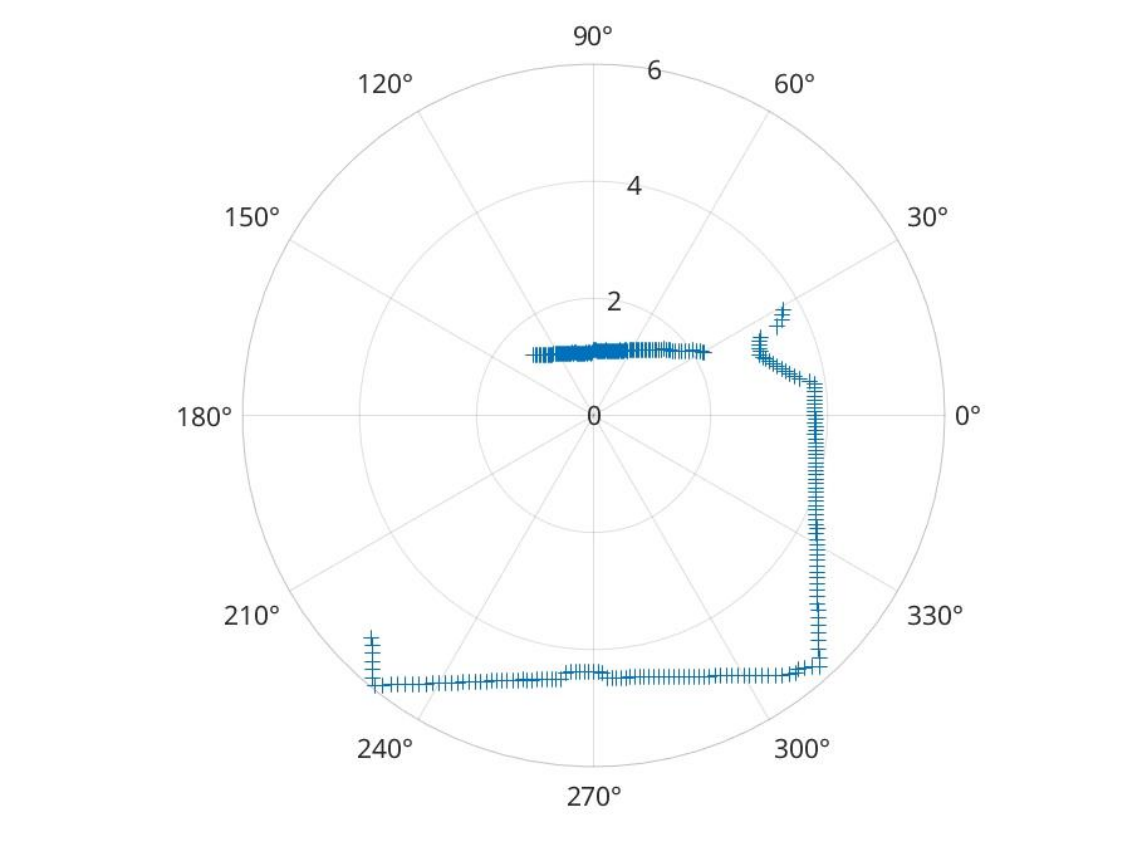
Non-Matching Features

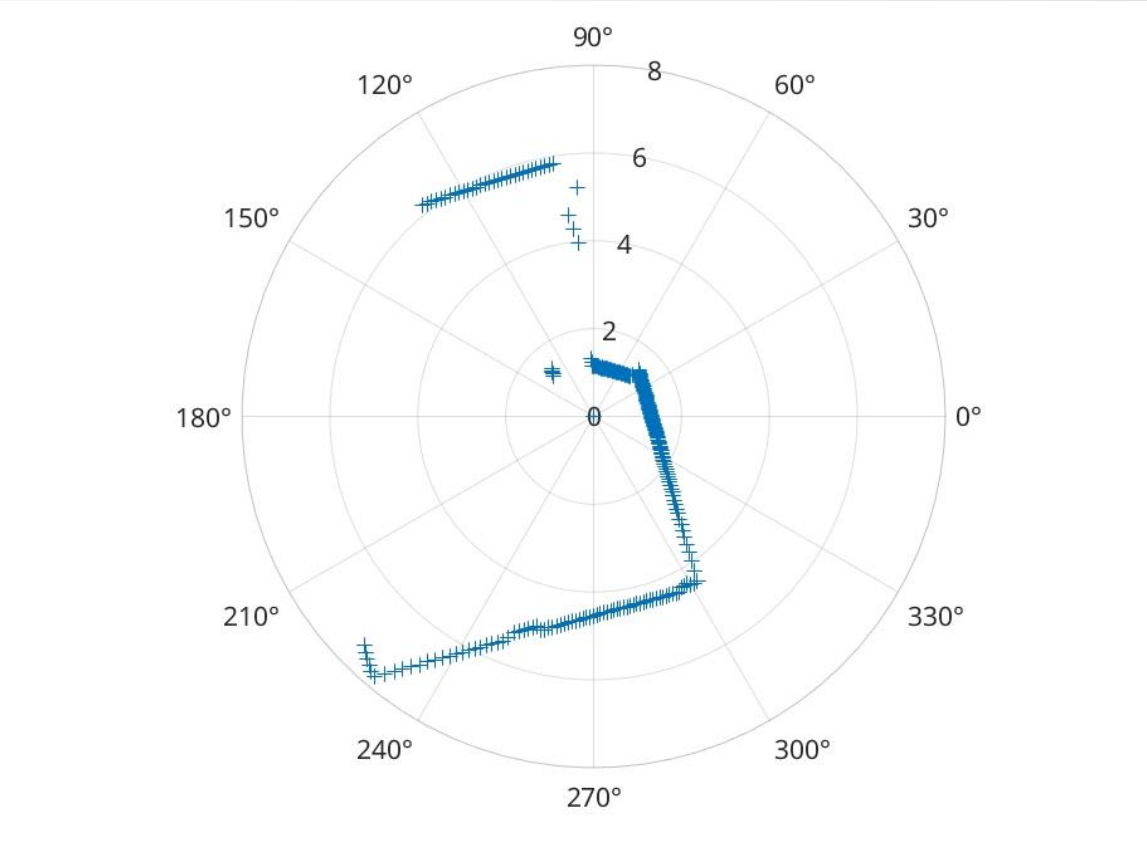
- The analysis so far has assumed that we have features which are common between the two scans
- However, this isn't guaranteed to be the case in general
- For example:
  - We could operate in environments without distinct features
  - We could operate in environments where the points don't even uniquely map to the same place
- Therefore, we need to have a generalized matching approach

Iterative Closest Point Scan Matching

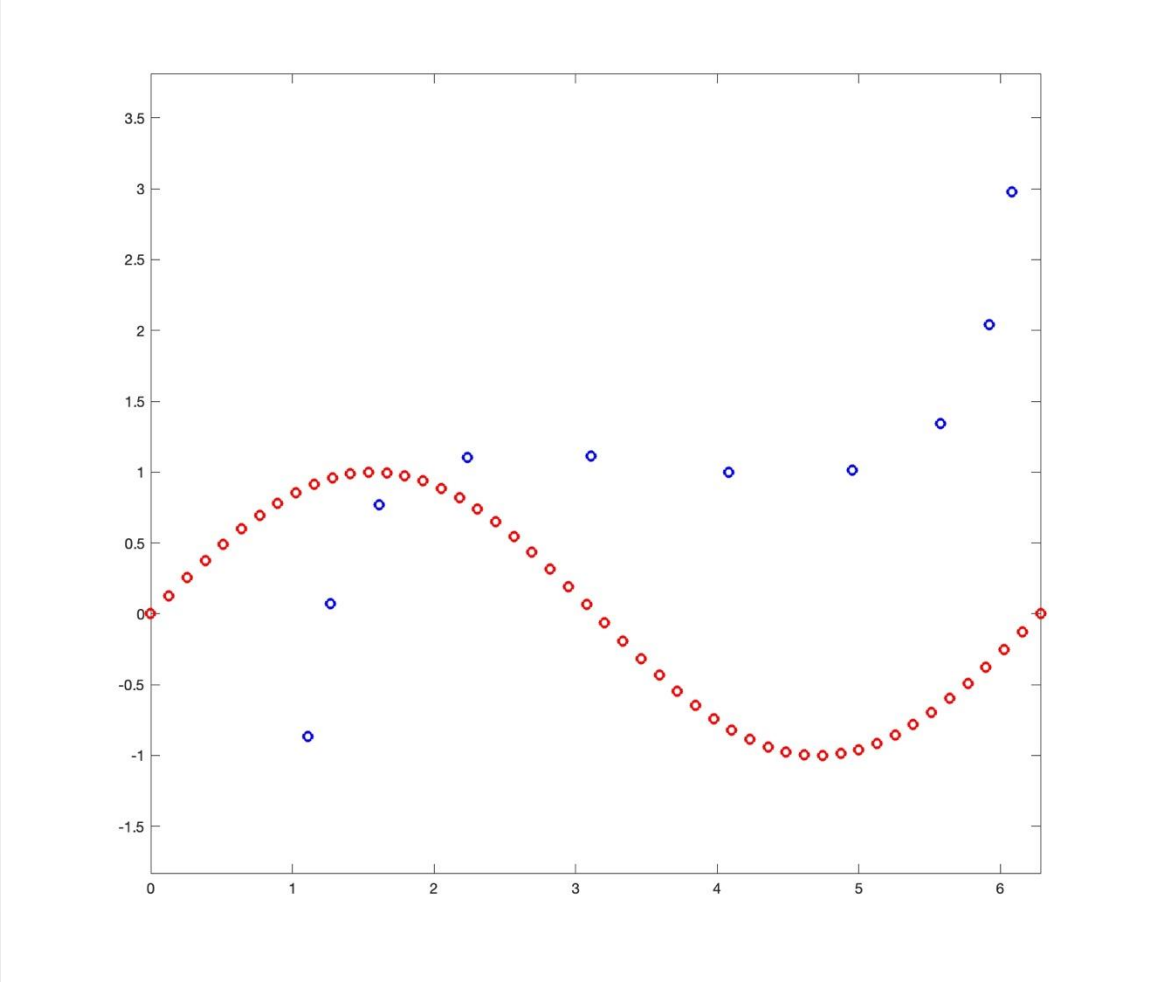
- *Why Do We Need Point Clouds?*
- *Depth Sensing Systems*
- *Overview of the Point Cloud SLAM*
- *Feature-Based Scan Matching*
- **Iterative Closest Point Scan Matching**
- *Point-to-Plane and Generalized ICP*
- *Loop Closure*

Iterative Closest Point Scan Matching





Simple Registration Example



Association Vector

$$O_k = \left\{ \mathbf{o}_k^1, \ldots, \mathbf{o}_k^{M_k} \right\}$$

$$I_k = \left\{ i_k^1,\ i_k^2,\ \ldots,\ i_k^{C_k} \right\}$$

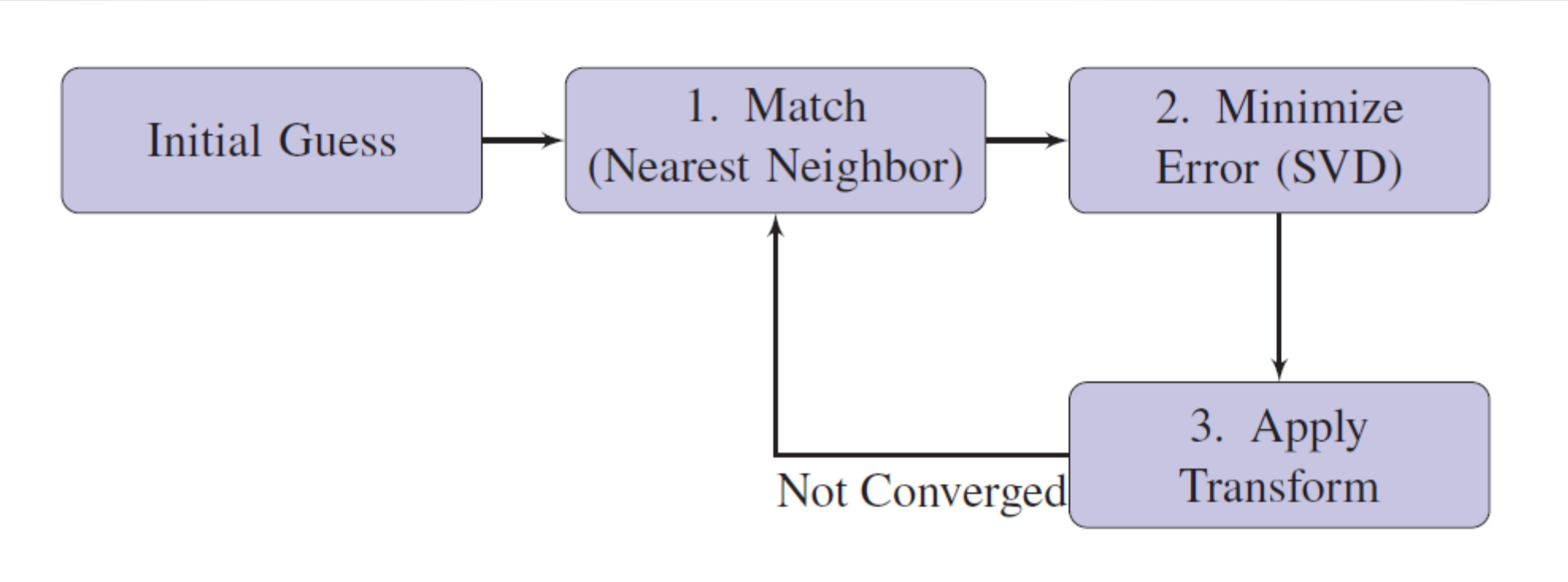
$$O_{k+1} = \left\{ \mathbf{o}_{k+1}^1, \ldots, \mathbf{o}_{k+1}^{M_{k+1}} \right\}$$

$$I_{k+1} = \left\{ i_{k+1}^1,\ i_{k+1}^2,\ \ldots,\ i_{k+1}^{C_k} \right\}$$

Matching (Computing the Association Vector)

- We make the following assumption:
  - Points in both scans which are close to one another are likely to be the same points

ICP Loop


Initial Guess
1. Match
(Nearest Neighbor)
2. Minimize
Error (SVD)
3. Apply
Transform
Not Converged

$\mathbf{T} \leftarrow \mathbf{T}^0$

repeat

  Initialize correspondence set $\boldsymbol{C} \leftarrow \emptyset$

  for each $\mathbf{o}_k^i \in \boldsymbol{O}_k$ **do**

    $j^* \leftarrow \arg\min_j \|\mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^j\|$

    if $\|\mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^{j^*}\| \leq d_{max}$ **then**

      Add correspondence pair (i, j^*) to $\boldsymbol{C}$

    end if

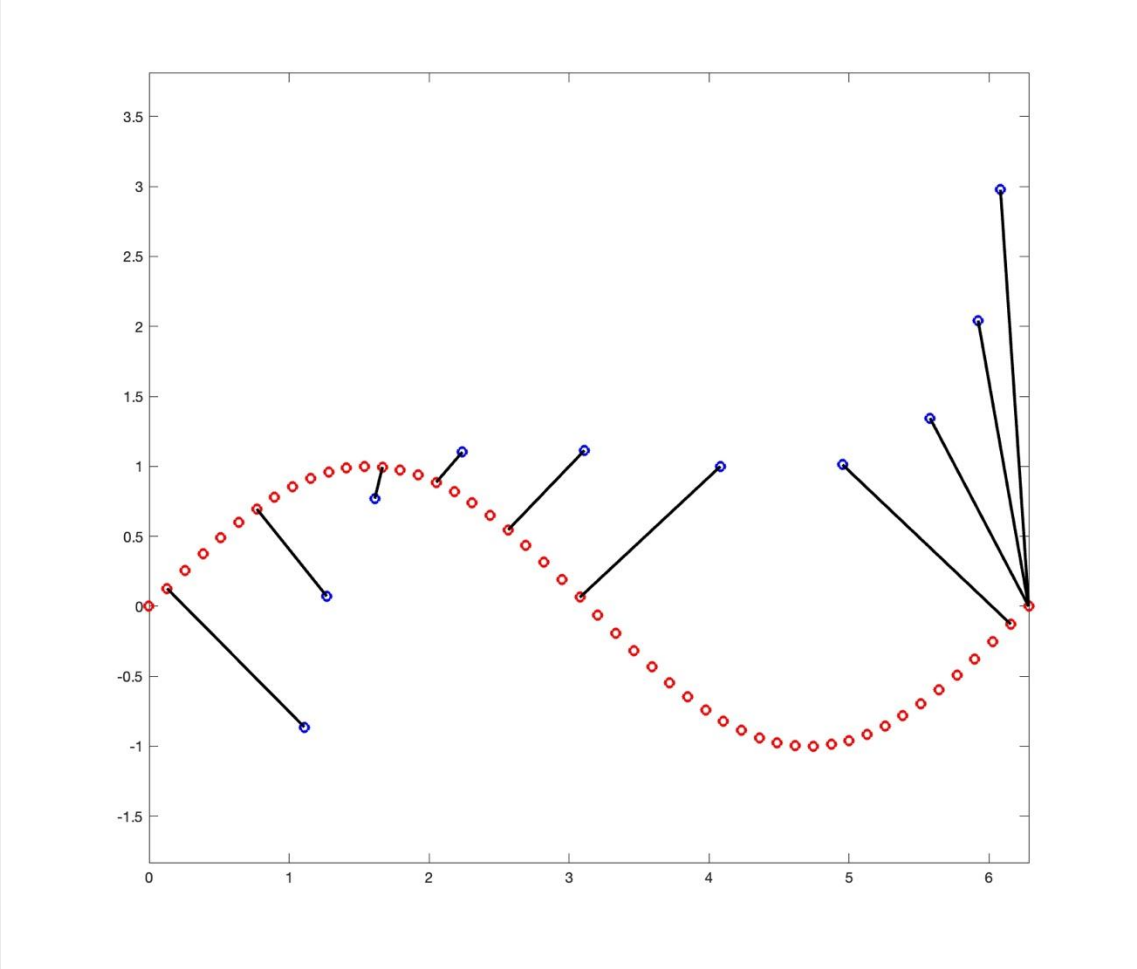
  end for

  $\mathbf{T} \leftarrow \arg\min_{\mathbf{T}} \sum_{(i,j^*)\in \boldsymbol{C}} \|\mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^{j^*}\|^2$

until Convergence (e.g., change in $\mathbf{T}$ is below threshold)

return $\mathbf{T}$

Matching Using Closest Point Heuristic



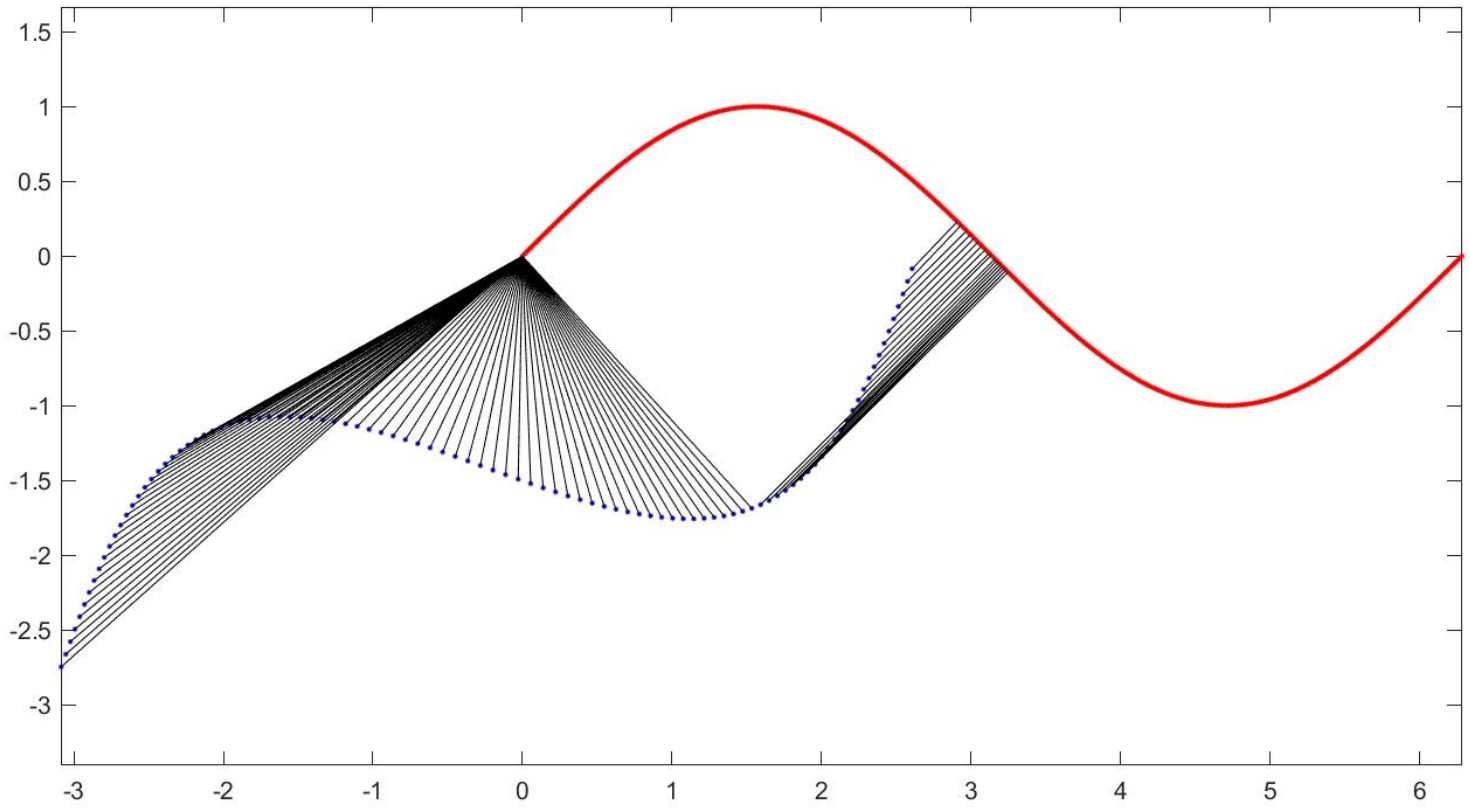
Minimize Error

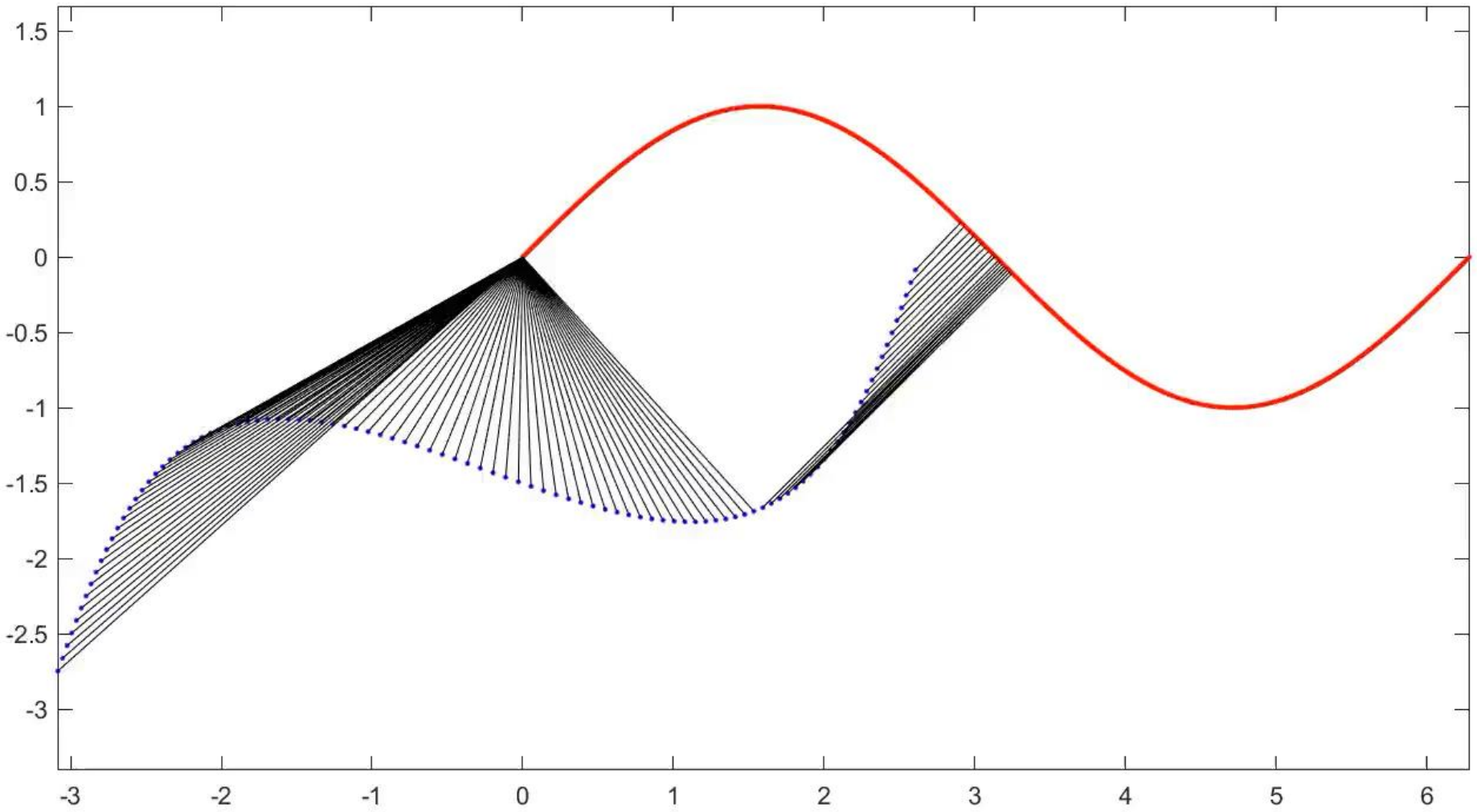
- We seek to compute the transformation which minimizes the alignment error

$$\Delta \mathrm{M}_{k,k+1}, \Delta \mathrm{t}_{k,k+1} = \arg\min_{\mathrm{M},\mathrm{t}} \sum_{j=1}^{M_k} ||\mathrm{o}_{k+1}^{j} - \mathrm{M}\mathrm{o}_{k}^{j} - \mathrm{t}||^2$$

-]We can use the Umeyama Alignment again

ICP Matching Example




1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2.5
-3
-3
-2
-1
0
1
2
3
4
5
6

Limitations of Point-to-Point ICP

- The point-to-point ICP method works as long as both scans contain measurements of the same points in the real-world
- However, as we've seen this might not be true for sensors such as LiDARs pointing at flat walls
- If the number of points is very small, ICP can get stuck in local minima
- An alternative is to use more general geometric properties such as planes

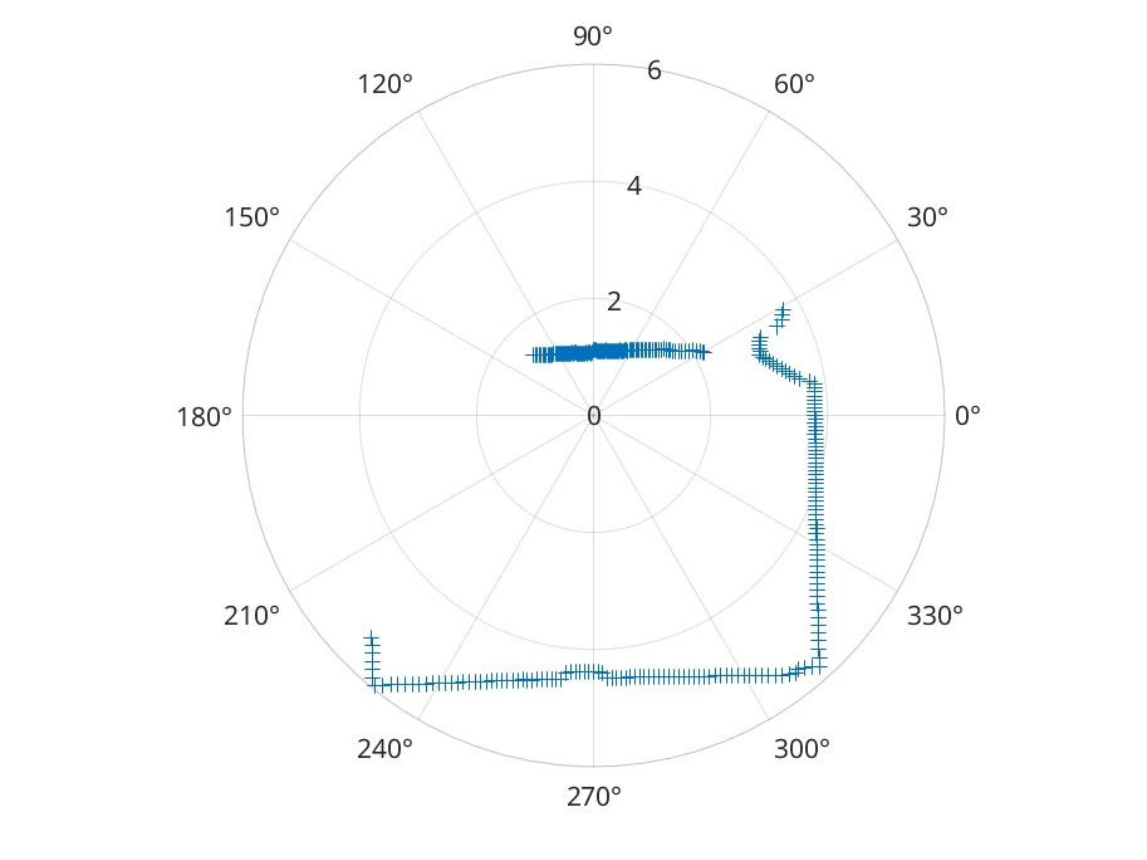
Point-to-Plane and Generalized ICP

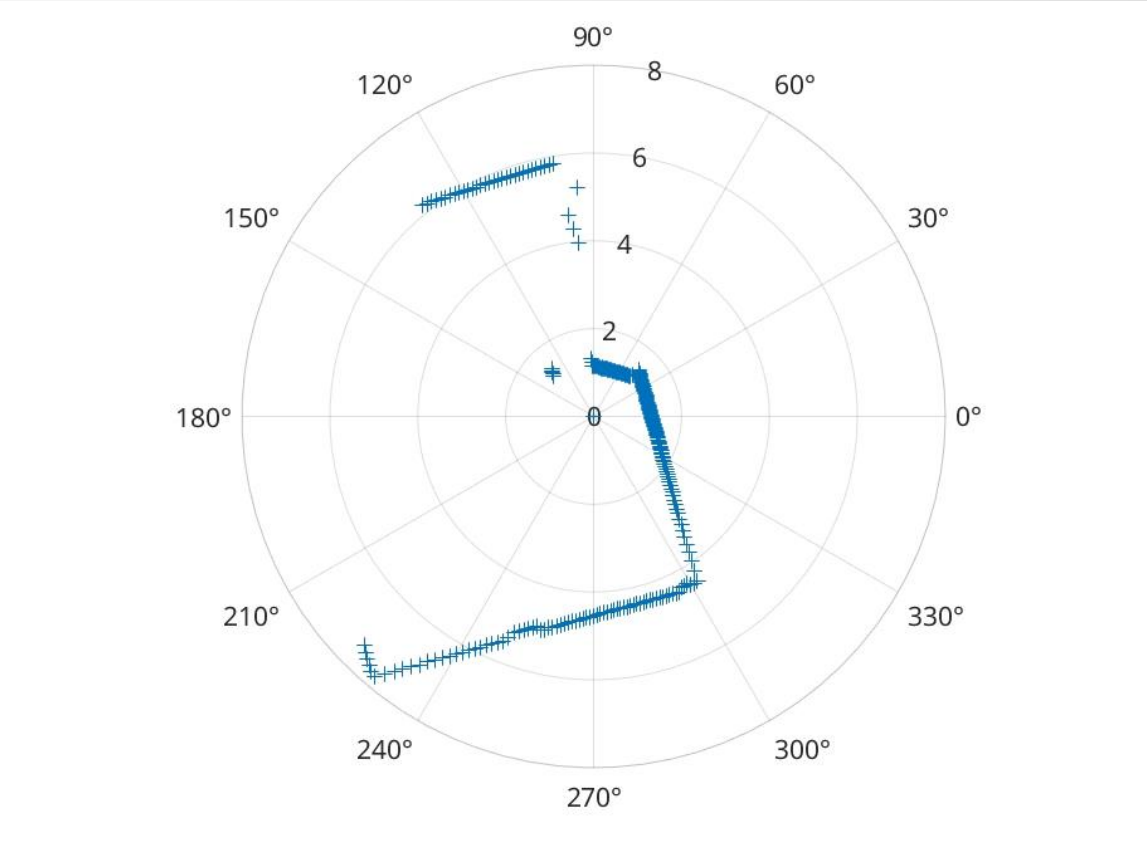
- *Why Do We Need Point Clouds?*
- *Depth Sensing Systems*
- *Overview of the Point Cloud SLAM*
- *Feature-Based Scan Matching*
- *Iterative Closest Point Scan Matching*
- Point-to-Plane and Generalized ICP
- *Loop Closure*

Point-to-Plane and Generalized ICP

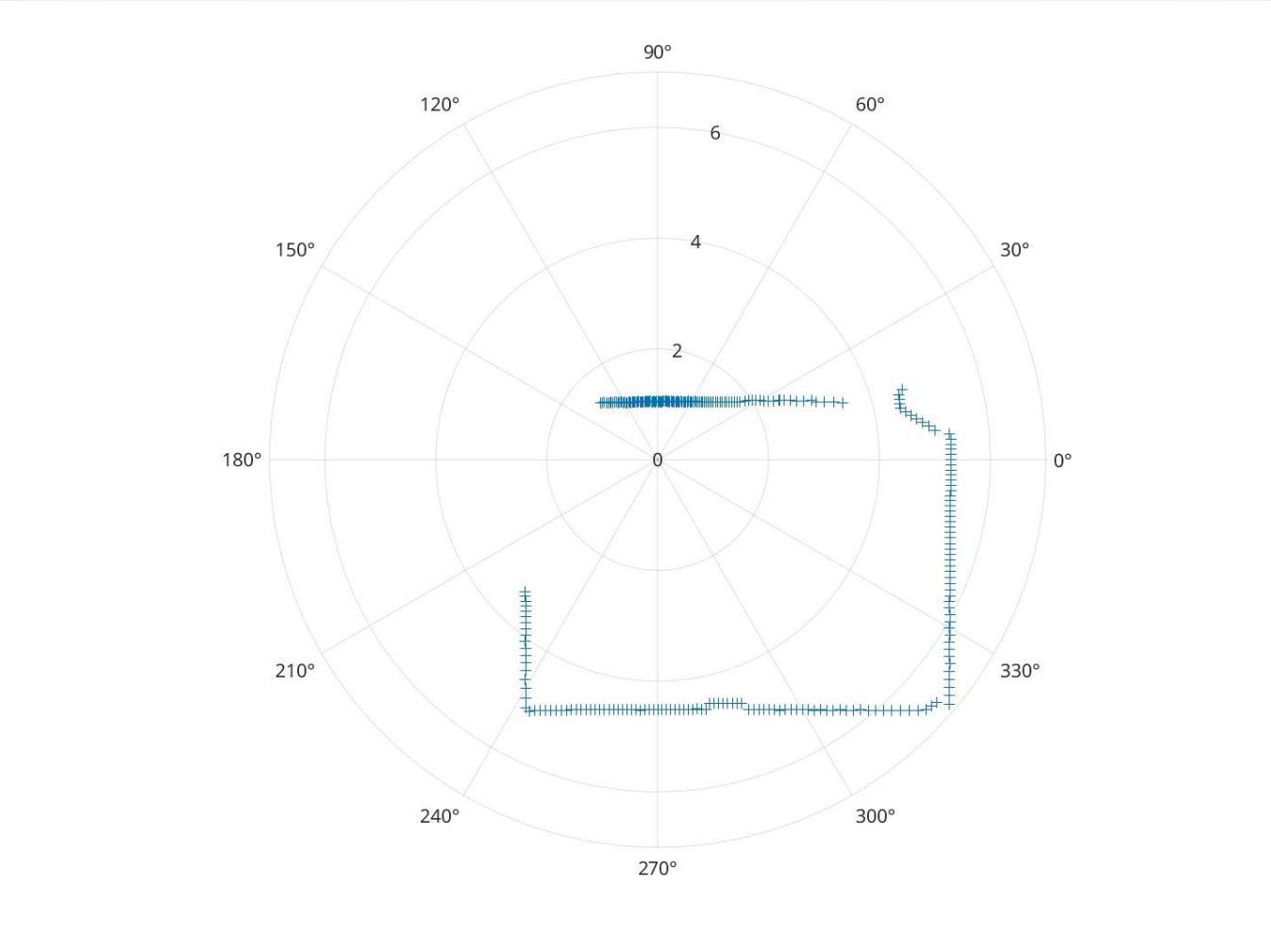
- In many environments we don’t have unique points that the point-to-point ICP can work with
- However, most environments possess alternative structures
- One very common example is planar surfaces, such as lines or walls

Scan Alignment Problem





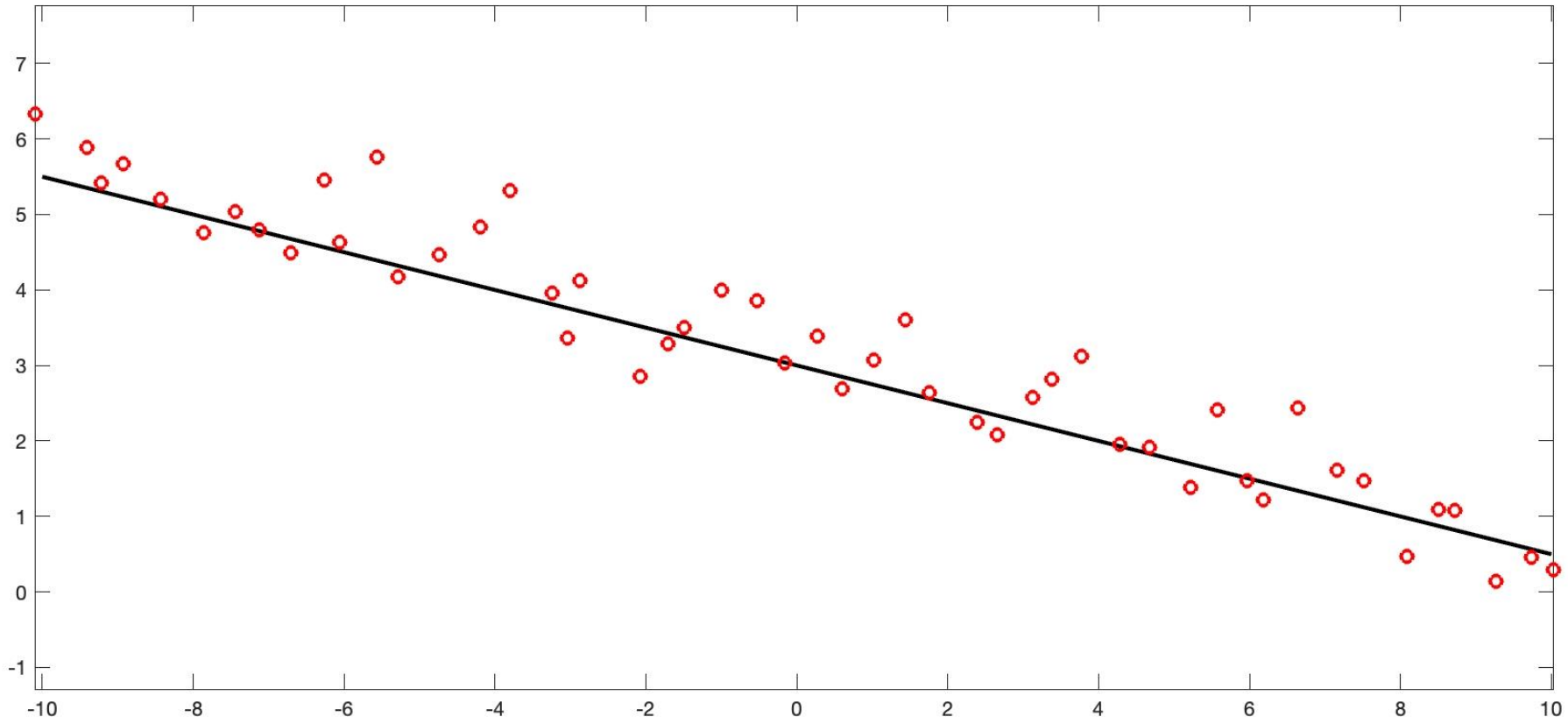
Point-to-Plane and Generalized ICP



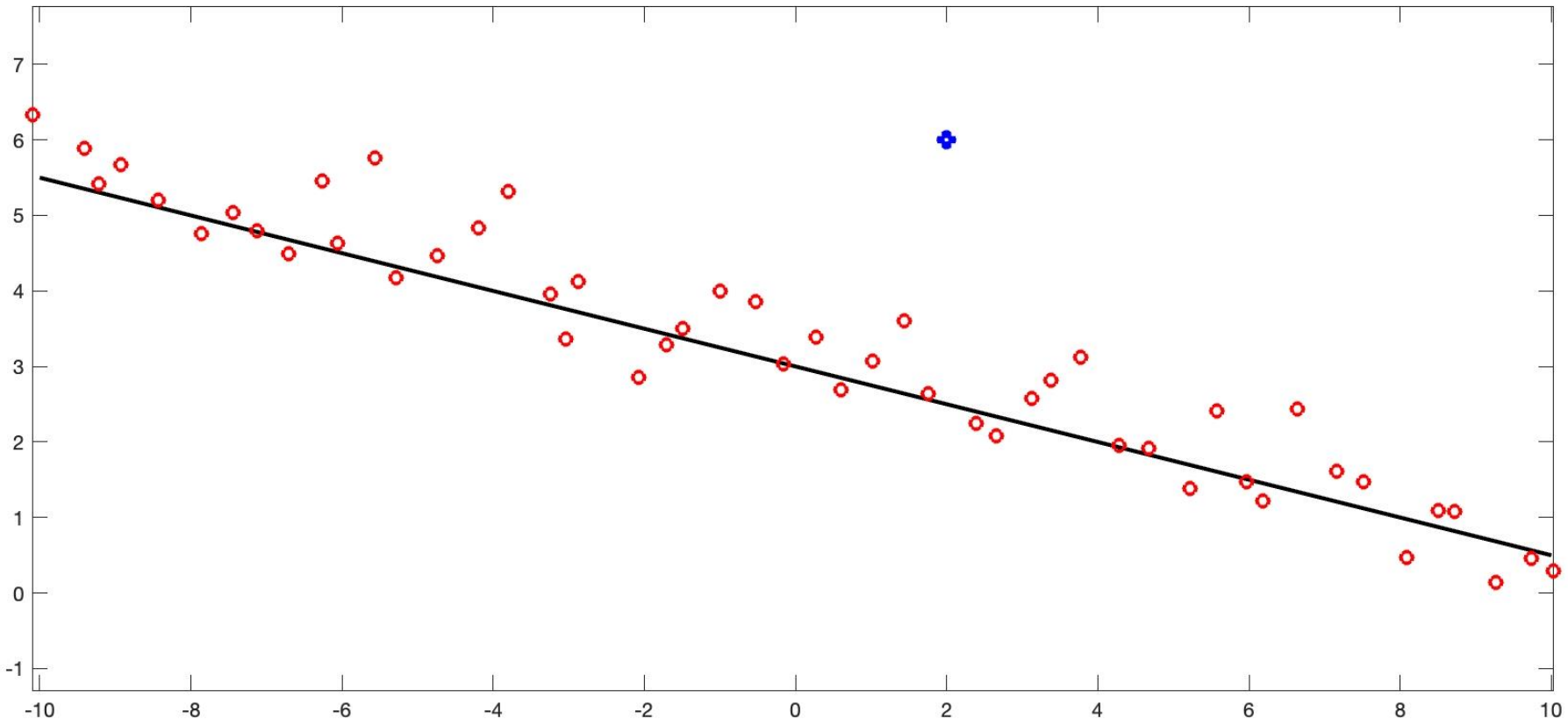
Point-to-Plane and Generalized ICP

- The idea is that a *subset* of the scan points will lie on planar surfaces
- Therefore, for the error metric we compute the orthogonal distance between the projected point and the planar surface

Noisy Wall



Noisy Wall Point Error



Plane Normal

- Suppose we match with ICP to the point

$$\mathbf{o}_{k+1}^{j}$$

- This projected point falls on a line whose unit normal vector is

$$\mathbf{n}_{k+1}^{j}$$

Use of the Plane Normal

- Using this, we modify the error function to be

$$\left\| \mathbf{n}_{k+1}^{j} \cdot \left(\mathbf{T}\mathbf{o}_{k}^{i} - \mathbf{o}_{k+1}^{j} \right) \right\|$$

$\mathbf{T} \leftarrow \mathbf{T}^0$

repeat

Initialize correspondence set $\boldsymbol{C} \leftarrow \emptyset$

for each $\mathbf{o}_k^i \in \boldsymbol{O}_k$ **do**

$j^* \leftarrow \arg\min_j \left\| \mathbf{n}_{k+1}^j \cdot \left(\mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^j \right) \right\|$

if $\| \mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^{j^*} \| \leq d_{max}$ **then**

Add correspondence pair (i, j^*) to $\boldsymbol{C}$

end if

end for

$\mathbf{T} \leftarrow \arg\min_{\mathbf{T}} \sum_{(i,j^*) \in \boldsymbol{C}} \left\| \mathbf{n}_{k+1}^j \cdot \left(\mathbf{T}\mathbf{o}_k^i - \mathbf{o}_{k+1}^j \right) \right\|^2$

until Convergence (e.g., change in $\mathbf{T}$ is below threshold)

return $\mathbf{T}$

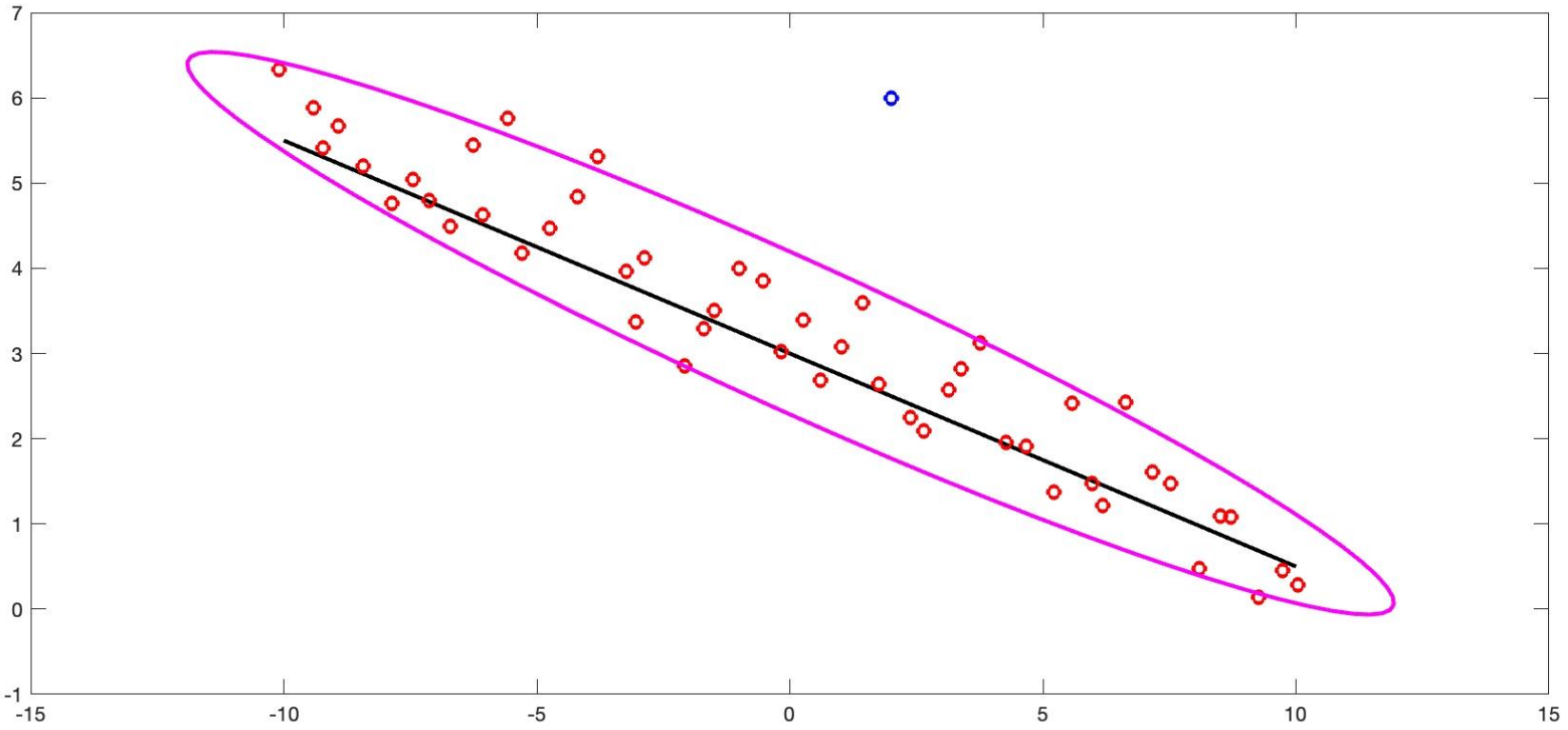
Computing the Plane Normal

- We compute the plane normal by taking the set of points in the neighbourhood of the point we want to match,

$$N\left(\mathbf{o}_{k+1}^{j}\right)=\left\{\left\|\mathbf{o}_{k+1}^{i}-\mathbf{o}_{k+1}^{j}\right\| \leq \epsilon\right\}$$

- The second moment of the distribution of the points can then be computed

Computing the Plane Normal



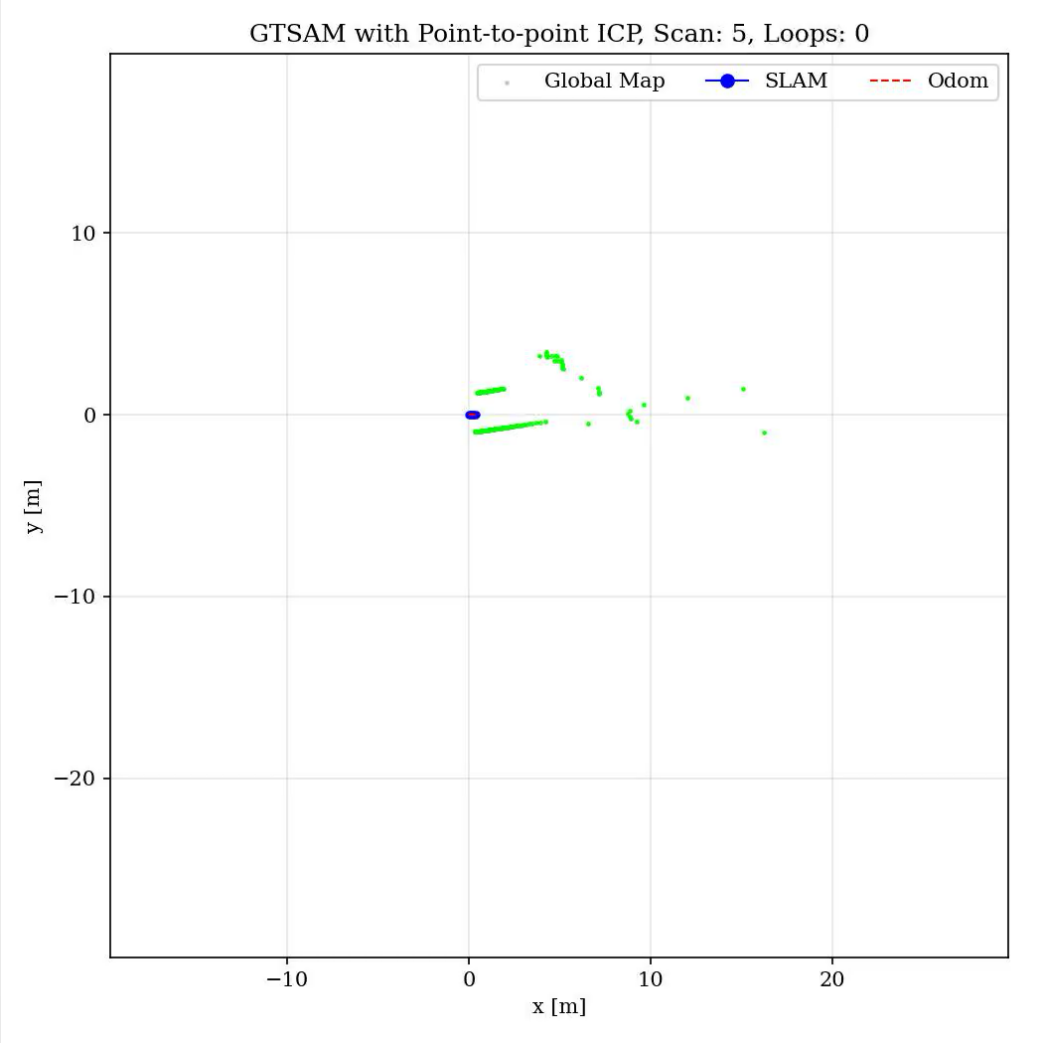
Optimizing the Solution

- **Unfortunately there is no closed form solution for the step**

$$\mathbf{T} \leftarrow \arg\min_{\mathbf{T}} \sum_{(i,j^*)\in C} \left\| \mathbf{n}_{k+1}^{j} \cdot \left(\mathbf{T}\mathbf{o}_{k}^{i} - \mathbf{o}_{k+1}^{j} \right) \right\|^2$$

- **It has to be solved numerically using techniques such as Levenberg-Marquardt**

Example


GTSAM with Point-to-point ICP, Scan: 5, Loops: 0
Global Map
SLAM
Odom
10
0
−10
−20
y [m]
−10
0
10
20
x [m]